VISVESVARAYA TECHNOLOGICAL UNIVERSITY "Jnana Sangama", Belagavi-590018



A Main Project Phase-II Report

on

"GESTURESPEAK: HAND GESTURE TO SENTENCE AND VOICE CONVERSION"

Submitted to

Visvesvaraya Technological University

In the partial fulfillment of requirements for the award of degree

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted By

[STUDENT NAME] 1	**[4RA23CS000]**
[STUDENT NAME] 2	**[4RA23CS000]**
[STUDENT NAME] 3	**[4RA23CS000]**

Under the guidance of

Prof. [GUIDE NAME]
Assistant Professor, Dept. of CSE
Rajeev Institute of Technology, Hassan





DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RAJEEV INSTITUTE OF TECHNOLOGY
HASSAN 573201

2026-27

RAJEEV INSTITUTE OF TECHNOLOGY, HASSAN

(Approved by AICTE, New Delhi and Affiliated to VTU, Belagavi.)

Plot # 1-D, Growth Centre, Industrial Area, B-M Bypass Road, Hassan-573201
Ph:(08172)-243180/80/84 Fax: (08172)-243183

Department of Computer Science & Engineering

CERTIFICATE

This is Certified that the Project entitled **"GESTURESPEAK: HAND GESTURE TO SENTENCE AND VOICE CONVERSION"** is carried out by **Mr./Ms. [STUDENT NAME] 1 [4RA23CS000], Mr./Ms. [STUDENT NAME] 2 [4RA23CS000], and Mr./Ms. [STUDENT NAME] 3 [4RA23CS000]**, are bonafide students of **RAJEEV INSTITUTE OF TECHNOLOGY, Hassan** in partial fulfilment for the award of **Bachelor of Engineering** in **Computer Science and Engineering** of **Visvesvaraya Technological University Belagavi,** during the year 2026-2027. The project report has been approved as it satisfies the academic requirements in respect of main-project prescribed for the said degree.

Prof. [GUIDE NAME]	**Dr. SURESHA D**	**Dr. MAHESH P K**
Assistant Professor Dept. of Computer Science & Engineering RIT, HASSAN	Head of the Department Computer Science & Engineering RIT, HASSAN	Principal RIT, HASSAN

DECLARATION

We, [STUDENT NAME] 1 [4RA23CS000], [STUDENT NAME] 2 [4RA23CS000], [STUDENT NAME] 3 [4RA23CS000], students of 7th sem B.E, in Computer Science and Engineering, Rajeev Institute of Technology, Hassan, hereby declare that the Project work being presented in the dissertation entitled “GestureSpeak: Hand Gesture to Sentence and Voice Conversion” has been carried out by us under the supervision of guide Prof. [GUIDE NAME], Assistant Professor, Department of Computer Science and Engineering, Rajeev Institute of Technology, Hassan, as partial fulfillment of requirement for the award of Degree Bachelor of Engineering in Computer Science and Engineering at Visvesvaraya Technological University, Belagavi is an authentic record of our work carried out by us during the academic year 2026-2027.

[STUDENT NAME] 1
4RA23CS000

[STUDENT NAME] 2
4RA23CS000

[STUDENT NAME] 3
4RA23CS000

PLACE: HASSAN
DATE:

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crowned our efforts with success.

We would like to profoundly thank the Management of Rajeev Institute of Technology and our President Dr. Rachana Rajeev for providing such a healthy environment.

We would like to express our sincere thanks to our Principal Dr. Mahesh P.K., Rajeev Institute of Technology, for his encouragement that motivated us for the successful completion of the Project work.

We wish to express our gratitude to Dr. Suresha D, Head of the Department of Computer Science & Engineering, for providing a good working environment and for his constant support and encouragement.

It gives us great pleasure to express our gratitude to Prof. [GUIDE NAME], Assistant Professor, Department of Computer Science & Engineering, for the expert guidance, initiative and encouragement that led us to complete this Project Work.

We would also like to thank all the staff members of the Computer Science and Engineering department who have directly or indirectly helped us in the successful completion of this Project Work, and we would also like to thank our parents.

[STUDENT NAME] 1 (4RA23CS000)
[STUDENT NAME] 2 (4RA23CS000)
[STUDENT NAME] 3 (4RA23CS000)

ABSTRACT

Communication is a basic human need, yet people who find it difficult to speak often depend on gestures that listeners may not understand. This project presents GestureSpeak, a computer-vision-based system that recognises a predefined set of hand gestures from a webcam and converts them into meaningful sentences and spoken output. The system is implemented in Python. OpenCV is used to capture and prepare video frames, and MediaPipe is used to detect the hand and extract its landmark coordinates. The landmark information is converted into a feature vector and given to a Random Forest classifier, which predicts the gesture label.

To make the output reliable, a confidence threshold and a stability (debounce) check of approximately one second are applied, so that momentary or uncertain predictions are not announced. The accepted gesture label is then mapped, through a CSV mapping file, to a sentence in English, Hindi or Kannada. The selected sentence is displayed in a Tkinter-based graphical interface and converted into speech using the gTTS or pyttsx3 text-to-speech engines. Separate data-collection and model-training scripts allow the gesture vocabulary and sentence mappings to be extended.

The report describes the theoretical background, requirements, architecture, module-level design, testing methodology and result-analysis procedure of the system. GestureSpeak is an academic prototype for a limited gesture vocabulary and not a complete sign-language translator, but it demonstrates how commonly available hardware and open-source libraries can be combined into an accessible, real-time, multilingual communication aid.

Keywords: *Hand gesture recognition, OpenCV, MediaPipe, Random Forest, text-to-speech, multilingual output, assistive communication.*

CONTENTS

Chapters **Page No.**

DECLARATION ..... **i**
ACKNOWLEDGEMENT ..... **ii**
ABSTRACT ..... **iii**
CONTENTS ..... **iv**
LIST OF FIGURES ..... **vi**
LIST OF TABLES ..... **vii**
1. INTRODUCTION **1**
1.1 Basics of the Project 1
1.2 Problem Statement 1
1.3 Motivation 2
1.4 Objectives of the Project 2
1.5 Scope of the Project 3
1.6 Existing System and Proposed System 4
1.7 Overview of Gesture-Based Communication 5
1.8 Methodology Overview 5
1.9 Advantages and Limitations 6
1.10 Organization of the Report 6
1.11 Summary of Introduction 7
2. LITERATURE SURVEY **8**
2.1 Introduction to the Literature Survey 8
2.2 Gesture Recognition: Background and Taxonomy 8
2.3 Related Work 9
2.4 Sensor-Based and Vision-Based Approaches 10
2.5 Computer Vision for Gesture Recognition 11
2.6 Image Acquisition and OpenCV 11
2.7 Hand Landmark Detection Using MediaPipe 12
2.8 Feature Extraction and Representation 14
2.9 Machine Learning for Gesture Classification 15
2.10 Decision Trees and Random Forest Classification 16
2.11 Evaluation Metrics for Classification 18
2.12 Sentence Mapping and Multilingual Output 19
2.13 Text-to-Speech Technologies 20
2.14 Graphical User Interfaces Using Tkinter 21
2.15 Research Gap 21
2.16 Summary of Literature Survey 22
3. SYSTEM REQUIREMENTS AND ARCHITECTURE **23**
3.1 Introduction 23
3.2 Hardware Requirements 23
3.3 Software Requirements 24
3.4 Functional Requirements 25
3.5 Non-Functional Requirements 26
3.6 Performance Requirements 26
3.7 Dataset and Model Requirements 27
3.8 Constraints and Limitations 28
3.9 System Architecture Overview 29
3.10 Summary of System Requirements 29
4. SYSTEM DESIGN AND IMPLEMENTATION **30**
4.1 Introduction 30

4.2 Architectural Design....30
4.3 Module Description....31
4.4 Data Flow Diagrams....32
4.5 Use-Case Diagram....32
4.6 Sequence of Operations....33
4.7 Webcam Capture and Frame Processing....33
4.8 Hand Landmark Extraction....34
4.9 Feature Preparation....35
4.10 Gesture Classification....35
4.11 Confidence Threshold and Gesture Stability....36
4.12 Sentence Mapping....36
4.13 Multilingual Sentence Selection....37
4.14 Text-to-Speech Implementation....37
4.15 Graphical User Interface....38
4.16 Data Collection and Model Training....39
4.17 Runtime Algorithm and Implementation Summary....40
4.18 Error Handling Strategy....40
4.19 Summary of System Design and Implementation....41
5. SOFTWARE TESTING....42
5.1 Introduction....42
5.2 Testing Objectives....42
5.3 Levels and Types of Testing....43
5.4 Unit Testing....43
5.5 Integration Testing....44
5.6 Functional and Usability Testing....44
5.7 Performance Testing....45
5.8 Test Environment....45
5.9 Test Cases....46
5.10 Testing Strategy and Summary....47
6. RESULTS AND DISCUSSION....48
6.1 Introduction....48
6.2 Application Execution....48
6.3 Webcam and Landmark Detection Results....49
6.4 Gesture Classification Results....49
6.5 Sentence Mapping and Language Results....50
6.6 Speech Output Results....51
6.7 Recognition Performance Evaluation....52
6.8 Discussion....53
6.9 Overall Result Analysis....54
6.10 Summary of Results....54
7. CONCLUSION AND FUTURE ENHANCEMENTS....55
7.1 Conclusion....55
7.2 Project Contributions and Limitations....55
7.3 Future Enhancements....56
7.4 Proposed Development Roadmap....57
REFERENCES....58

LIST OF FIGURES

Fig. No.	Figure Name	Page No.
1.1	Typical hand-gesture-to-speech communication scenario	5
1.2	Methodology followed in the project	6
2.1	Classification of hand-gesture recognition approaches	9
2.2	The 21 hand landmarks detected by MediaPipe Hands	13
2.3	Structure of a Random Forest classifier	17
2.4	General stages of a text-to-speech system	20
3.1	Block diagram of the GestureSpeak processing pipeline	29
4.1	Architectural design of GestureSpeak	31
4.2	Level-0 data flow diagram (context diagram)	32
4.3	Level-1 data flow diagram	32
4.4	Use-case diagram of GestureSpeak	33
4.5	Sequence diagram for one recognition cycle	33
4.6	Live webcam frame in the application window	34
4.7	Detected hand with landmarks and connections	34
4.8	Flowchart of the confidence threshold and stability check	36
4.9	Interface showing a recognised sentence and the selected language	38
4.10	Main window of the GestureSpeak application	39
5.1	Levels of testing applied to GestureSpeak	43
6.1	GestureSpeak application after successful launch	49
6.2	Hand with detected landmark points	49
6.3	Gesture prediction and recognised label	50
6.4	Sentence displayed with English output selected	50
6.5	Sentence displayed with Hindi output selected	51
6.6	Sentence displayed with Kannada output selected	51
6.7	Recognised sentence during speech output	52
6.8	Confusion matrix of the trained classifier on the test set	53

LIST OF TABLES

Table No.	Table Name	Page No.
1.1	Comparison of Communication Approaches	4
2.1	Summary of Related Work	10
2.2	MediaPipe Hand Landmarks and Their Indices	13
2.3	Qualitative Comparison of Classification Methods	18
2.4	Structure of a Confusion Matrix for Two Classes	19
2.5	Comparison of the Text-to-Speech Options Used in the Project	21
3.1	Recommended Hardware Configuration	24
3.2	Software and Libraries Used	24
3.3	Functional Requirements of GestureSpeak	26
3.4	Gesture Vocabulary and Associated Sentences	28
4.1	Modules and Files of GestureSpeak	31
4.2	Logical Structure of the Sentence Mapping File	37
4.3	Error Conditions and Expected Handling	41
5.1	Performance Measurements	45
5.2	Test Environment Details	46
5.3	Test Cases and Test Results	47
6.1	Sentence Mapping Observations	51
6.2	Speech Output Observations	52
6.3	Per-Class Recognition Performance	53

CHAPTER 1

INTRODUCTION

1.1 Basics of the Project

GestureSpeak is a computer-vision-based application designed to recognise predefined hand gestures and convert them into meaningful sentences and spoken output. Human communication often depends on speech, but people who have difficulty speaking may prefer to communicate through gestures. Interpreting gestures manually requires the other person to understand their meaning. This may create communication difficulties in situations where the listener is unfamiliar with the gestures.

Recent developments in computer vision and machine learning have made it possible for computers to identify patterns in images and video. A webcam can capture hand movements, image-processing techniques can locate the hand, and a trained machine-learning model can classify the extracted features [9], [10]. GestureSpeak combines these techniques to map recognised gestures to predefined sentences and provide speech output.

The project uses Python as its primary programming language. OpenCV [1] supports webcam capture and image processing, MediaPipe [2], [3] extracts hand landmarks, and a Random Forest classifier [4] predicts the gesture class. A sentence-mapping component retrieves the corresponding text in English, Hindi, or Kannada. A text-to-speech component converts the selected sentence into audio.

The main objective is to demonstrate an accessible, real-time communication aid using commonly available hardware and software. The complete application can be executed on an ordinary computer with a webcam and does not require a specialised glove, depth camera or dedicated hand-tracking device.

At a conceptual level, the system performs a chain of transformations. A physical hand pose is first converted into a digital image, the image is converted into a set of landmark coordinates, the coordinates are converted into a gesture label, the label is converted into a sentence in the chosen language, and finally the sentence is converted into sound. Each of these conversions is handled by a separate software component, which keeps the design simple to understand and easy to extend.

1.2 Problem Statement

Communication is an essential part of everyday life. People communicate their needs, feelings, questions, and instructions through speech, writing, facial expressions, and body movements. However, when two people do not share an effective communication method, even simple activities such as asking for help, requesting an object, or communicating an emergency may become difficult.

A person who uses gestures may need the listener to interpret each gesture correctly. If the listener is unfamiliar with the gesture system, the intended message may not be understood. Written communication may not always be convenient, particularly when a quick response is required or when typing is difficult.

The problem addressed by GestureSpeak is the conversion of a limited set of predefined hand gestures into understandable text and spoken sentences. The system should capture a hand through a webcam, extract relevant features, classify the gesture, retrieve its associated sentence, and generate speech in the selected language.

The system is not intended to recognise every sign language or interpret unrestricted conversations. Instead, it provides a controlled demonstration of gesture-based communication that can be extended with additional gestures and better training data.

1.3 Motivation

The motivation for GestureSpeak comes from the need to make digital communication more accessible. Assistive technologies can help reduce barriers between people by converting one form of information into another. Speech-to-text, text-to-speech, screen readers, and gesture recognition are examples of technologies that support different communication needs.

Hand gestures are useful for this project because they can be captured using an ordinary webcam. A dedicated sensor is not necessary for the basic implementation. This reduces the initial hardware cost and makes the system easier to demonstrate in a laboratory or classroom.

Another motivation is the practical application of machine learning. Instead of studying classification algorithms only in theory, the project connects them to a complete application. The student can understand image acquisition, feature extraction, model training, prediction, data mapping, graphical user interfaces, and audio generation within one system.

Multilingual output is another useful feature. A gesture can be associated with sentences in English, Hindi, and Kannada. This demonstrates how the same recognised input can support different language preferences without training a separate gesture classifier for each language. In a multilingual country, the ability to speak a recognised message in a regional language increases the usefulness of such a tool.

1.4 Objectives of the Project

The primary objective of GestureSpeak is to develop a working prototype that converts predefined hand gestures into sentences and speech. The system combines computer vision, machine learning, language mapping, and audio generation in a single application.

The specific objectives are:

1. To capture hand gestures through a webcam using Python and OpenCV.
2. To detect the hand and extract its landmark coordinates using MediaPipe.
3. To use landmark information as input features for a Random Forest classifier.
4. To map the predicted gesture label to a predefined sentence.
5. To provide sentence output in English, Hindi, and Kannada.
6. To convert the selected sentence into spoken audio using the configured text-to-speech engine.
7. To provide a graphical interface for interacting with the application.
8. To reduce accidental repeated predictions by checking whether a gesture remains stable for a specified period.
9. To allow the gesture dataset and sentence mappings to be extended.
10. To evaluate the system through functional testing and practical webcam trials.

These objectives establish the functional boundaries of the project and provide a basis for designing the system architecture and evaluating its behaviour.

1.5 Scope of the Project

The scope of GestureSpeak is limited to a predefined vocabulary of hand gestures. The project README describes 15 predefined gestures. The recognised gesture labels are associated with sentences through a mapping file, allowing the same classification output to produce different language versions of a sentence.

The system includes webcam-based image acquisition, hand-landmark extraction, gesture classification, sentence mapping, multilingual text output, and speech generation. It also includes a data-collection facility and a model-training script to support further development.

The application is suitable for academic demonstrations, prototype development, and learning how computer vision can be integrated with a machine-learning model. It may also serve as a foundation for future assistive communication systems.

However, the project does not implement unrestricted sign-language translation. It does not understand all hand movements, facial expressions, grammar, or the complete context of a conversation. Its performance depends on the gestures represented in its training data, the visibility of the hand, lighting conditions, camera quality, and the accuracy of the trained classifier.

The application should therefore be described as a predefined-gesture communication prototype, not as a complete replacement for a human interpreter.

1.6 Existing System and Proposed System

1.6.1 Existing System

In a manual gesture-communication process, the listener observes a gesture and interprets its meaning using prior knowledge. If the listener does not understand the gesture, communication may fail. Written communication is another option, but it may require additional time and may not be convenient in every situation.

Some gesture-recognition systems use specialised sensors, gloves, or dedicated hardware. These devices can provide useful information about hand movement, but they may increase cost and reduce convenience. Other systems use image classification, yet they may be limited to a particular dataset, language, or set of gestures.

1.6.2 Proposed System

GestureSpeak uses a webcam to capture the hand and computer-vision techniques to extract landmark information. The classifier predicts a predefined gesture label, and the sentence mapper retrieves the associated message. The selected message can then be displayed and spoken in the chosen language.

The proposed approach uses commonly available software libraries and can operate without a specialised hand-tracking device. The design is modular, so the classifier, sentence mappings, and speech engine can be improved separately.

The main difference is the integration of recognition, sentence conversion, and speech output into one application. Table 1.1 summarises the differences between the three approaches discussed above.

Aspect	Manual interpretation	Sensor / glove based	GestureSpeak (proposed)
Input device	None (human observer)	Gloves or dedicated sensors	Ordinary webcam
Listener must know gestures	Yes	No, if output is converted	No, output is sentence and speech
Additional hardware cost	None	Higher	Low (existing webcam)
Convenience for user	Depends on listener	Device must be worn	No device worn on the hand
Language output	Spoken by the listener	Depends on the system	English, Hindi and Kannada
Vocabulary	Open, but needs trained listener	System dependent	Predefined, extendable

Table 1.1: Comparison of Communication Approaches

1.7 Overview of Gesture-Based Communication

A gesture is a movement or posture of the body, most often of the hands and arms, that carries information. People use gestures alongside speech to emphasise a point, to point at an object, or to signal agreement and refusal. In some situations gestures replace speech completely, as in sign languages used by deaf communities, which are complete languages with their own vocabulary and grammar [9], [11].

From a computing point of view, hand gestures are commonly divided into two groups. A **static gesture** is a fixed hand posture that can be identified from a single image, for example a closed fist or an open palm. A **dynamic gesture** involves movement over time, such as waving, and must be recognised from a sequence of frames. GestureSpeak concentrates on static hand postures because they can be classified frame by frame from landmark coordinates and are suitable for a controlled vocabulary of short messages.

Gesture recognition is an important topic in human–computer interaction because it allows a person to communicate with a machine in a natural way, without a keyboard or mouse. Vision-based gesture recognition has been widely surveyed in the literature, and the surveys identify the main stages of such systems as image acquisition, hand detection, feature extraction and classification [9], [10]. The same stages form the backbone of GestureSpeak, with two additional stages that convert the classification result into a sentence and into speech.

[Space reserved for Figure 1.1]

Insert an illustrative picture or diagram: a user showing a hand gesture to the webcam, the computer displaying the sentence, and the loudspeaker producing speech for a listener.

Figure 1.1: Typical hand-gesture-to-speech communication scenario

1.8 Methodology Overview

The project was carried out as a sequence of connected phases. Each phase produces an output that is used by the next phase.

- **Requirement study:** the communication problem, the list of predefined gestures and the target languages were identified, and suitable software libraries were selected.
- **Data collection:** hand-landmark examples were recorded for every gesture class using the data-collection module and stored with their gesture labels.
- **Model training:** the labelled landmark features were used to train a Random Forest classifier, and the trained model and label encoder were saved to files.

- **Integration:** the webcam module, landmark extractor, classifier, sentence mapper and speech module were combined and connected to a Tkinter interface.
- **Testing and evaluation:** the modules and the complete application were tested using the test cases described in Chapter 5, and the observed behaviour was analysed in Chapter 6.

[Space reserved for Figure 1.2]

Insert a flow diagram with the phases: Requirement Study → Data Collection → Feature Preparation → Model Training → Integration → Testing and Evaluation.

Figure 1.2: Methodology followed in the project

1.9 Advantages and Limitations

GestureSpeak provides several advantages. It uses a webcam rather than requiring a specialised gesture sensor, combines multiple software components into one application, and supports sentence output in three languages. The separation of landmark extraction, classification, sentence mapping, and speech generation also makes the code easier to maintain.

The project has limitations. The classifier can only reliably recognise gesture classes represented in its training data. Poor lighting, a partially hidden hand, changes in camera angle, and differences between users may affect predictions. A classifier may also confuse gestures that produce similar landmark patterns. Speech generation may depend on the availability of the selected speech engine and its language support.

The project should not be treated as a medical or safety-critical communication device without extensive additional evaluation. It is an academic prototype whose performance must be measured through systematic testing.

1.10 Organization of the Report

The remaining part of the report is organised into six chapters:

- **Chapter 2 – Literature Survey** reviews the technical foundations and related work: gesture recognition, OpenCV, MediaPipe, feature extraction, Random Forest, evaluation metrics, sentence mapping, text-to-speech and graphical interfaces.
- **Chapter 3 – System Requirements and Architecture** lists the hardware, software, functional, non-functional, performance and dataset requirements, and gives an overview of the architecture.
- **Chapter 4 – System Design and Implementation** describes the modular design, the data flow, the working of every module, and the runtime algorithm.

- **Chapter 5 – Software Testing** explains the testing objectives, levels of testing, the test environment and the test cases.
- **Chapter 6 – Results and Discussion** presents the application outputs and explains how recognition performance is evaluated.
- **Chapter 7 – Conclusion and Future Enhancements** summarises the work, its limitations and possible extensions.

1.11 Summary of Introduction

This chapter introduced GestureSpeak, described the communication problem, explained the motivation and objectives, defined the project scope, and compared the proposed approach with manual and sensor-based alternatives. It also gave a short overview of gesture-based communication, the methodology followed, and the organisation of the report. The next chapter reviews the technical foundations and related work.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction to the Literature Survey

A literature survey examines existing research, tools, and methods related to the problem being addressed. It helps identify the techniques commonly used in a field, understand their advantages and limitations, and determine how a proposed project can be designed.

Gesture recognition combines computer vision, feature extraction, classification, and sometimes natural-language processing. Researchers have explored image-based recognition, sensor-based recognition, landmark-based representations, deep-learning models, and conventional machine-learning algorithms. Each approach has different computational requirements and practical limitations.

For GestureSpeak, the literature review focuses on technologies that are directly related to the implementation: image acquisition using OpenCV, hand-landmark detection using MediaPipe, classification using Random Forest, and conversion of text to speech. These technologies form a pipeline in which each stage transforms information for the next stage.

The review also considers the distinction between recognising isolated predefined gestures and translating a complete sign language. This distinction is important because recognising a limited gesture vocabulary is a more restricted problem than understanding continuous gestures, grammar, facial expressions, and conversational context.

2.2 Gesture Recognition: Background and Taxonomy

Gesture recognition is the process of interpreting human gestures with a computer so that they can be used as input or converted into another form of information. Surveys of the field describe gestures made with the hands, arms, face, head and whole body, and discuss applications such as sign-language interpretation, human–computer interaction, robot control, virtual reality and assistive technology [9], [10].

Gestures can be classified in several ways. According to **time behaviour**, they are static or dynamic. Static gestures are hand postures that remain fixed, while dynamic gestures are defined by a trajectory or sequence of postures. According to **purpose**, they may be communicative, such as a sign that stands for a word, or manipulative, such as a movement used to drag a virtual object. According to **vocabulary**, a system may recognise isolated gestures, where each gesture is a separate unit, or continuous gestures, where signs flow into one another and the system must also find the boundaries between them.

A predefined-gesture system, such as GestureSpeak, recognises isolated static gestures from a closed vocabulary. This is much simpler than full sign-language recognition, because it does not require a grammar, temporal segmentation or facial-expression analysis. It is

nevertheless useful for short, frequently needed messages. Sign languages are natural languages with their own grammar and are not simply gestures for letters or words; a gesture vocabulary used in a prototype should therefore not be described as a sign language [11].

[Space reserved for Figure 2.1]

Insert a tree diagram: Hand Gesture Recognition → Sensor-based (gloves, motion sensors) / Vision-based (image-based, landmark-based); and Gestures → Static / Dynamic, Isolated / Continuous. Highlight the branch used by GestureSpeak.

Figure 2.1: Classification of hand-gesture recognition approaches

2.3 Related Work

Several surveys and research papers provide the background for this project. The most relevant works are summarised below and in Table 2.1.

Mitra and Acharya reviewed gesture recognition in general [9]. Their survey covers the recognition of hand and arm gestures as well as face, head and body gestures and discusses tools such as hidden Markov models, finite-state machines and neural networks. It shows that gesture recognition is a broad problem and that the choice of method depends strongly on the type of gesture and the intended application.

Rautaray and Agrawal surveyed vision-based hand gesture recognition for human–computer interaction [10]. They organise a vision-based system into stages: gesture acquisition, detection and segmentation of the hand, feature extraction and recognition. They also discuss environmental constraints such as background, lighting and camera position, which continue to affect practical systems.

Cheok, Omar and Jaward reviewed hand gesture and sign language recognition techniques [11]. Their review follows the same chain of data acquisition, pre-processing, segmentation, feature extraction and classification and compares the classifiers used for gesture and sign recognition. It also draws attention to the difference between recognising isolated signs and recognising continuous sign-language sentences.

Starner and Pentland presented one of the early real-time systems for recognising American Sign Language from video using hidden Markov models [12]. Their work used a limited vocabulary and showed that a camera alone can provide enough information to recognise signs without wearing a data glove. It is an example of the vision-based approach followed by GestureSpeak, although GestureSpeak recognises only isolated static postures.

Zhang et al. described MediaPipe Hands, an on-device real-time hand-tracking solution that estimates 21 three-dimensional hand landmarks from a single image [3]. It is built on the MediaPipe framework introduced by **Lugaresi et al.** [2], in which a perception task is expressed as a graph of processing components. The availability of an accurate, ready-made hand-landmark detector greatly simplifies gesture-recognition projects, because the developer can concentrate on classification instead of building a hand detector from the beginning.

Breiman introduced Random Forests, an ensemble of tree predictors in which every tree depends on a randomly selected subset of the training data and features [4]. The paper shows that the generalisation error of a forest converges as the number of trees increases and that the method is robust to noise. Random Forests are implemented in the scikit-learn library described by **Pedregosa et al.** [7], which is used in this project.

Ref.	Authors / Year	Focus of the work	Relevance to GestureSpeak
[9]	Mitra and Acharya, 2007	Survey of gesture recognition methods and applications	Background on gesture types and recognition tools
[10]	Rautaray and Agrawal, 2015	Survey of vision-based hand gesture recognition for HCI	Stage-wise structure of a vision-based system
[11]	Cheok et al., 2019	Review of hand gesture and sign language recognition	Pre-processing, feature extraction and classifier choices
[12]	Starner and Pentland, 1995	Real-time ASL recognition from video using HMMs	Camera-only recognition of a limited vocabulary
[2]	Lugaresi et al., 2019	MediaPipe framework for perception pipelines	Framework underlying the hand-tracking component
[3]	Zhang et al., 2020	On-device real-time hand tracking (21 landmarks)	Source of the landmark features used by the classifier
[4]	Breiman, 2001	Random Forests	Classification algorithm used in the project
[7]	Pedregosa et al., 2011	Scikit-learn: machine learning in Python	Implementation library for the classifier

Table 2.1: Summary of Related Work

2.4 Sensor-Based and Vision-Based Approaches

Gesture-recognition systems are usually divided into sensor-based and vision-based systems [10], [11]. A **sensor-based** system uses devices worn on the hand, such as data gloves with bend sensors, accelerometers or gyroscopes. These devices measure finger bending and hand motion directly, so the data is precise and does not depend on lighting. However, the user must wear the device, the equipment adds cost and the hand may feel restricted.

A **vision-based** system uses one or more cameras and image processing to find the hand and interpret its posture. The user is not required to wear anything, and an ordinary webcam may be sufficient. The price of this convenience is that the accuracy depends on the quality of the image: lighting, background, camera angle, motion blur and occlusion of fingers can all reduce reliability.

Vision-based systems may classify the whole image directly or first extract a structured representation of the hand. GestureSpeak is a vision-based system of the second type. It uses MediaPipe to convert each frame into landmark coordinates, which removes most of the background information before classification and makes the classifier smaller and faster than an image-based network.

2.5 Computer Vision for Gesture Recognition

Computer vision is a field of artificial intelligence concerned with extracting useful information from images and videos. It enables a computer to identify objects, track movement, detect shapes, and interpret visual patterns [13]. In gesture recognition, computer vision is used to locate the hand and represent its position and configuration.

A basic image-based approach may classify the entire image directly. Such a system can learn useful patterns, but it may also become sensitive to the background, clothing, lighting, and camera position. If the background changes significantly, the model may learn features unrelated to the gesture itself.

A landmark-based approach attempts to reduce this problem by representing the hand using selected points, such as fingertips, joints, and the wrist. Instead of processing every pixel as a classification feature, the system uses a structured representation of the hand.

GestureSpeak follows the landmark-based approach. The webcam provides frames, MediaPipe estimates hand landmarks, and the extracted information is passed to the classifier. This separation allows the system to concentrate more directly on hand geometry.

Relevance to the project: Computer vision supplies the visual input and provides the foundation for the remaining processing stages.

2.6 Image Acquisition and OpenCV

OpenCV is an open-source computer-vision library that provides functions for reading images, capturing video, changing colour spaces, resizing frames, drawing shapes, and displaying processed images [1], [16]. It is widely used in educational and practical computer-vision applications.

In a webcam-based application, the first task is to establish a connection with the camera. The application repeatedly reads frames and passes them to the processing pipeline. A frame

is a single image captured from the video stream. Processing successive frames allows the system to respond to a user's changing hand position.

Image acquisition must be handled carefully. The program should check whether the camera opens successfully and whether a frame is actually received. If the camera is unavailable, the application should show a meaningful message rather than continuing with invalid image data.

OpenCV can also be used to display the recognised gesture and related information on the application window. This visual feedback helps the user understand what the system has detected and supports debugging during development.

2.6.1 Digital Images, Frames and Colour Spaces

A digital image is a two-dimensional array of pixels, and a colour image stores several values for each pixel, normally one each for red, green and blue [13]. Video is a sequence of such images, called frames, captured at a certain number of frames per second (FPS). The delay between capturing a frame and obtaining its result is important for a real-time application, since it determines how quickly the user receives feedback.

OpenCV stores colour images in blue–green–red (BGR) order, whereas many other libraries, including MediaPipe, expect red–green–blue (RGB) order. A conversion between the two is therefore required before a frame is passed to such a library. If the conversion is forgotten, the image still looks plausible but the colours are swapped, and the quality of detection may fall without any error message. In many webcam applications the frame is also flipped horizontally so that the display behaves like a mirror, which is more natural for the user.

In GestureSpeak, OpenCV is the interface between the physical webcam and the software pipeline. The quality and consistency of the captured frames influence the reliability of the later landmark-detection and classification stages.

2.7 Hand Landmark Detection Using MediaPipe

MediaPipe is a framework for building perception pipelines [2]. Its hand-tracking solution estimates a structured set of hand landmarks from an image [3], [17]. These landmarks represent important points on the hand, including the wrist, finger joints, and fingertips.

A landmark-based representation provides more structured information than a raw image alone. The system can use the relative positions of the landmarks to distinguish hand configurations. For example, a finger being extended or bent changes the relative locations of its joints and fingertip.

MediaPipe Hands is designed to support hand landmark estimation from images and video. It combines detection and tracking processes to maintain hand information across frames.

The quality of the estimated landmarks can vary depending on the visibility of the hand, occlusion, lighting, and camera angle.

2.7.1 Two-Stage Hand Tracking Pipeline

The hand-tracking solution described by Zhang et al. consists of two cooperating models [3]. The first is a **palm detector**, which finds the hand in the full image and produces a bounding region. The second is a **hand-landmark model**, which works on the cropped hand region and predicts the landmark coordinates. In a video, the position of the hand in the previous frame is used to predict where the hand will be in the next frame, so the palm detector does not have to run on every frame. It is run again only when the hand is lost. This design keeps the computation low enough for real-time execution on ordinary computers.

2.7.2 The 21 Hand Landmarks

MediaPipe Hands returns 21 landmarks for every detected hand. Each landmark has x and y coordinates that are normalised to the range 0 to 1 by the image width and height, and a z value that represents relative depth, with the wrist as the reference point [3], [17]. The landmarks are always returned in the same order, which is what makes them suitable as classifier features. Table 2.2 lists the index and name of each landmark.

Index	Landmark	Index	Landmark
0	WRIST	11	MIDDLE_FINGER_DIP
1	THUMB_CMC	12	MIDDLE_FINGER_TIP
2	THUMB_MCP	13	RING_FINGER_MCP
3	THUMB_IP	14	RING_FINGER_PIP
4	THUMB_TIP	15	RING_FINGER_DIP
5	INDEX_FINGER_MCP	16	RING_FINGER_TIP
6	INDEX_FINGER_PIP	17	PINKY_MCP
7	INDEX_FINGER_DIP	18	PINKY_PIP
8	INDEX_FINGER_TIP	19	PINKY_DIP
9	MIDDLE_FINGER_MCP	20	PINKY_TIP
10	MIDDLE_FINGER_PIP		

Table 2.2: MediaPipe Hand Landmarks and Their Indices

[Space reserved for Figure 2.2]

Insert the standard hand-landmark diagram showing points 0 to 20 on a hand, with the wrist, thumb, index, middle, ring and pinky landmarks labelled.

Figure 2.2: The 21 hand landmarks detected by MediaPipe Hands

In GestureSpeak, MediaPipe extracts the hand landmarks before classification. The resulting landmark coordinates are transformed into the feature representation expected by the trained classifier. Consistency between the training features and the features generated during live prediction is essential. If the order or format of the landmarks changes, the classifier may receive incorrect input.

2.8 Feature Extraction and Representation

Feature extraction transforms raw input into a representation that a machine-learning algorithm can use. For hand gesture recognition, features may include landmark coordinates, distances between joints, angles between fingers, or other geometric relationships.

Raw image pixels contain large amounts of information, including background objects and illumination differences. Landmark features provide a compact representation of the hand structure. This can reduce the amount of information that must be handled by a conventional classifier.

However, feature extraction does not automatically guarantee correct recognition. If two gestures produce similar landmark patterns, the classifier may find them difficult to separate. Inconsistent camera framing or different hand orientations may also change the feature values.

2.8.1 Normalisation of Landmark Coordinates

Landmark coordinates measured in the image depend on where the hand is placed in the frame and how close it is to the camera. The same gesture shown at the left and at the right of the image would therefore produce different coordinate values. A common remedy is to express every landmark relative to a reference landmark, usually the wrist, and to divide by a scale factor s, such as the distance between the wrist and the base of the middle finger [10], [14]. This is shown in Equation 2.1, where x_0 is the coordinate of the reference landmark.

(2.1)

Such a transformation makes the feature vector less dependent on position and distance. Whether a particular implementation applies it, or uses raw coordinates, is a design decision that must be applied identically during data collection, training and live prediction.

The training process and live prediction process must use the same feature format. The same number of features, ordering of landmarks, preprocessing steps, and coordinate conventions should be maintained. Otherwise, a model trained on one representation may produce unreliable predictions when it receives another.

GestureSpeak uses landmark information as the classifier's input. The feature-extraction stage therefore acts as a bridge between the visual hand-detection process and the machine-learning model.

2.9 Machine Learning for Gesture Classification

Machine learning enables a computer to learn patterns from examples rather than relying entirely on manually written rules. In supervised learning, each training example contains input features and a known class label. The algorithm learns relationships between the features and labels and then predicts labels for new examples [14].

Gesture recognition can be treated as a classification problem. Each gesture is represented by its landmark features, and its label identifies the gesture category. During training, the model receives labelled examples. During prediction, it receives new landmark features and estimates the corresponding class.

A conventional machine-learning classifier can be appropriate when the input has already been converted into a compact feature vector. It may require fewer computational resources than a large image-based neural network, although the actual requirements depend on the dataset and implementation.

The quality of the model depends on the quantity, diversity, and correctness of the training data. If the dataset contains only a small number of examples from one person, the model may not generalise well to other users. A separate test set and careful evaluation are therefore important.

2.9.1 Training, Testing and Generalisation

A supervised model is built in two phases. In the **training phase**, the algorithm adjusts its internal parameters so that its predictions match the known labels of the training examples. In the **testing phase**, the finished model is applied to examples it has never seen, and its predictions are compared with the true labels. The performance on unseen examples measures how well the model has generalised. For this reason, the available data is divided into a training set and a test set, and the test set is not used during training [14].

A model that fits the training data very closely but performs poorly on new data is said to **overfit**. Overfitting is more likely when the dataset is small, when it contains little variety, or when the model is very complex. The opposite problem, **underfitting**, occurs when the model is too simple to capture the pattern. Good practice is to collect varied data, to keep a separate test set and, where possible, to use cross-validation, in which the data is divided into several parts and each part is used in turn for testing.

GestureSpeak uses a Random Forest classifier for the gesture-label prediction stage.

2.10 Decision Trees and Random Forest Classification

Random Forest is an ensemble-learning algorithm that combines multiple decision trees [4]. A decision tree divides the feature space using a sequence of conditions. For example, a tree may learn that certain relationships between hand landmarks are associated with one gesture rather than another.

A Random Forest trains multiple trees using randomised samples and feature selections. For classification, the trees contribute their predictions, and the ensemble determines the final class. Combining multiple trees can make the prediction less dependent on the decisions of a single tree.

Random Forest is useful for structured feature data and can model non-linear relationships between features and class labels. It also supports multi-class classification, which is required when the system recognises more than two gestures.

2.10.1 Decision Trees

A decision tree is built by repeatedly splitting the training data. At each internal node, the algorithm selects a feature and a threshold that separate the examples into two groups that are as pure as possible, meaning that each group contains mainly one class. The splitting continues until the nodes are pure or a stopping condition, such as a maximum depth, is reached. A new example is classified by following the conditions from the root of the tree to a leaf, and the class of the leaf is the prediction.

The quality of a split is measured by an impurity function. Two commonly used measures are the Gini impurity and the entropy [14]. For a node t in which a fraction p_k of the examples belong to class k out of K classes, they are defined as follows.

(2.2)

(2.3)

Both measures are zero for a pure node and largest when all classes are equally mixed. The split that gives the largest reduction in impurity is chosen. The reduction in entropy is called the information gain. A single deep tree can fit the training data very closely but is sensitive to small changes in the data, which is a form of overfitting.

2.10.2 Bagging and Random Feature Selection

Random Forest reduces the instability of a single tree by combining two ideas. The first is **bagging** (bootstrap aggregating) [5]: every tree is trained on a bootstrap sample, which is a random sample of the training set drawn with replacement. Because each sample is different, the trees differ from each other. The second idea is **random feature selection** [4], [6]: at each split, only a randomly chosen subset of the features is considered. This prevents

a few strong features from dominating every tree and further increases the diversity of the trees.

For classification, each of the T trees in the forest votes for a class, and the class receiving the most votes becomes the prediction, as in Equation 2.4, where h_t(x) is the prediction of the t-th tree for the feature vector x.

(2.4)

In the scikit-learn implementation, the class probabilities of the trees are averaged, as in Equation 2.5, and the class with the highest averaged probability is returned [7], [18]. The averaged value is often used as a confidence-like score. It is a measure of agreement among the trees and not a guarantee that the prediction is correct.

(2.5)

2.10.3 Important Parameters and Advantages

Some important parameters of a Random Forest classifier are the number of trees (`n_estimators`), the maximum depth of each tree (`max_depth`), the number of features considered at each split (`max_features`) and the random seed (`random_state`) that makes the training repeatable [18]. Increasing the number of trees generally stabilises the result at the cost of a larger model and slightly slower prediction.

Random Forest has several advantages for a project such as GestureSpeak. It works well on numeric feature vectors of moderate size, handles several classes naturally, needs little tuning, does not require the features to be scaled, and can report which features were most useful for the decision. Prediction on a single feature vector is fast, which is important when a prediction is made for every video frame.

The method has limitations. It cannot compensate for missing or poorly labelled training examples, and its predictions may be unreliable when live features differ greatly from the training distribution. It also cannot extrapolate beyond the range of the training data. The classifier must therefore be trained using representative examples and evaluated on data that was not used for fitting.

[Space reserved for Figure 2.3]

Insert a diagram showing a feature vector given to several decision trees (Tree 1, Tree 2, … Tree T), each producing a class prediction, followed by a majority-voting block that outputs the final gesture label.

Figure 2.3: Structure of a Random Forest classifier

2.10.4 Comparison with Other Classifiers

Other classifiers can also be applied to landmark features. A support vector machine (SVM) finds a boundary that separates the classes with the largest margin [8]. The k-nearest-neighbour method classifies an example by the majority class among the closest training examples. Convolutional neural networks learn features directly from images and are the usual choice when raw images are classified [14]. Table 2.3 gives a qualitative comparison. The choice made in this project reflects the compact landmark input, the need for fast prediction on ordinary hardware and the simplicity of training.

Method	Typical input	Main strengths	Main limitations
Decision tree	Feature vector	Easy to understand, fast	Sensitive to small data changes
Random Forest	Feature vector	Stable, handles many classes, little tuning	Larger model than a single tree; cannot extrapolate
SVM	Feature vector	Effective margin-based separation	Needs feature scaling; kernel and parameter choice
k-NN	Feature vector	Simple, no training phase	Slower prediction with large data
CNN	Raw image	Learns features automatically	Needs more data and computation

Table 2.3: Qualitative Comparison of Classification Methods

2.11 Evaluation Metrics for Classification

The performance of a classifier is measured by comparing its predictions with the known labels of a test set. For a given class, the predictions fall into four groups: true positives (TP), where the class is predicted correctly; true negatives (TN), where an example of another class is correctly rejected; false positives (FP), where an example of another class is wrongly predicted as this class; and false negatives (FN), where an example of this class is predicted as another class [14]. These counts are arranged in a **confusion matrix**, whose rows represent the true classes and whose columns represent the predicted classes. In a multi-class problem such as gesture recognition, the matrix has one row and one column for each gesture, and the off-diagonal entries show which gestures are confused with each other.

The commonly used metrics are defined in Equations 2.6 to 2.9.

(2.6)

(2.7)

(2.8)

(2.9)

Accuracy is the fraction of all predictions that are correct. **Precision** tells how many of the examples predicted as a class really belong to it, and **recall** tells how many examples of the class were found. The **F1-score** is the harmonic mean of precision and recall. Accuracy alone can hide poor results for some classes, especially when the classes are unequal in size, so per-class precision, recall and F1-score and the confusion matrix should be reported together.

	Predicted: Positive	**Predicted: Negative**
Actual: Positive	True Positive (TP)	False Negative (FN)
Actual: Negative	False Positive (FP)	True Negative (TN)

Table 2.4: Structure of a Confusion Matrix for Two Classes

2.12 Sentence Mapping and Multilingual Output

Gesture recognition and sentence generation are separate tasks. The classifier identifies a gesture label, but the label alone may not be meaningful to a user. A mapping component connects each recognised label to a readable sentence.

A mapping table can contain a gesture identifier and its sentence in several languages. When the classifier predicts a label, the program searches the mapping data for the corresponding record. The selected language determines which sentence is displayed and passed to the speech engine.

Keeping sentences in a separate CSV file provides flexibility. A sentence can be edited without retraining the gesture classifier, provided the gesture label remains unchanged. Additional language columns can also be added if the mapping and speech engine support them.

However, every classifier label should have a valid mapping entry. Missing or duplicate labels may lead to empty output or incorrect messages. The program should handle an unknown label gracefully and should not assume that every possible classifier output has a valid sentence.

2.12.1 Character Encoding for Indian Languages

Hindi is written in the Devanagari script and Kannada in the Kannada script. Text in these scripts can be stored and displayed correctly only if a Unicode encoding is used. A CSV file containing Hindi and Kannada sentences should therefore be saved in UTF-8 and read with the same encoding. If a different encoding is assumed, the sentences appear as meaningless symbols, and the speech engine receives incorrect text. The graphical interface must also use a font that contains the glyphs of the required scripts.

GestureSpeak separates gesture classification from sentence mapping, which improves maintainability and makes the vocabulary easier to extend.

2.13 Text-to-Speech Technologies

Text-to-speech (TTS) converts written text into audio. A TTS system accepts a string and generates spoken output. This can make an application more accessible when a user needs to communicate a message without speaking it directly [15].

GestureSpeak uses gTTS [21] and pyttsx3 [22] as speech-generation options. gTTS commonly relies on an online service, while pyttsx3 can use locally available speech engines. Their behaviour, voice quality, and language support depend on the installed environment and the selected voice.

2.13.1 Stages of a TTS System

A TTS system generally has two main parts [15]. The **text-analysis stage** converts the input into a form suitable for pronunciation: it separates the text into words and sentences, expands numbers and abbreviations, and converts words into phoneme sequences with the help of pronunciation rules or dictionaries. The **speech-synthesis stage** then generates the audio waveform. Older systems joined recorded units of speech (concatenative synthesis) or generated the waveform from parameters (parametric synthesis), while modern systems commonly use neural networks to produce more natural voices. The application that uses TTS does not need to implement these steps, because they are hidden inside the engine.

[Space reserved for Figure 2.4]

Insert a block diagram: Input Text → Text Analysis (normalisation, phonemes) → Speech Synthesis (waveform generation) → Audio Output.

Figure 2.4: General stages of a text-to-speech system

2.13.2 gTTS and pyttsx3

gTTS is a Python library that sends the text to an online speech service and receives the audio in return, so it needs an internet connection [21]. pyttsx3 is an offline library that provides one programming interface over the speech engines installed in the operating system [22]; the voices that are available, and therefore the supported languages, depend on the computer. Table 2.5 compares the two options.

Aspect	gTTS	pyttsx3
Operation	Online speech service	Locally installed speech engine
Internet requirement	Required	Not required
Language and voice availability	Depends on the online service	Depends on voices installed on the computer
Typical output	Audio returned by the service	Audio produced by the local engine

Aspect	gTTS	pyttsx3
Main risk	Failure when the network is unavailable	Required language voice may be missing

Table 2.5: Comparison of the Text-to-Speech Options Used in the Project

Speech generation should be separated from gesture recognition. The classifier should first produce a label, the mapping component should retrieve the sentence, and only then should the speech engine receive the final text. This organisation makes it easier to diagnose whether a problem originates in recognition, mapping, or audio output.

The application should also handle speech-generation errors. For example, an online TTS request may fail without internet access, and a local engine may not have the required language voice installed. The project's use of two TTS options demonstrates the importance of considering both online convenience and local availability.

2.14 Graphical User Interfaces Using Tkinter

Tkinter is the standard graphical-user-interface toolkit of Python and provides widgets such as windows, labels, buttons, drop-down lists and canvases [20]. A Tkinter program is **event driven**: after the window is created, the program enters a main loop that waits for events such as button clicks and calls the functions registered for them.

This has an important consequence for a real-time application. If a long operation, such as waiting for speech output, is executed directly inside an event handler, the main loop is blocked and the window stops responding until the operation finishes. Long operations are therefore usually executed in a separate thread, or the interface is refreshed periodically using a timer function. A camera application also has to convert the OpenCV frame into an image format that Tkinter can display on a label or canvas.

2.15 Research Gap

The surveys and tools reviewed in this chapter show that the individual technologies needed for gesture-to-speech conversion are mature. Vision-based recognition has been studied for many years [9], [10], [11]; accurate hand-landmark detection is available as a ready-made component [3]; and classifiers and speech engines are available as open-source libraries [7], [21], [22]. Much of the research effort, however, concentrates on the recognition stage and on improving the accuracy of classification.

The gap addressed by GestureSpeak is the integration of these stages into a single, simple application that runs on an ordinary computer with a webcam. The project combines landmark-based recognition, a confidence threshold with a stability check, an editable multilingual sentence table and speech output in English, Hindi and Kannada. The multilingual part is of particular practical interest in the Indian context, where a recognised message may need to be spoken in a regional language.

The project also keeps the recognition vocabulary separate from the sentence vocabulary so that messages can be edited without retraining the classifier. It does not claim to solve the wider problems of continuous sign-language translation, user-independent recognition or recognition in unconstrained environments, which remain open research problems.

2.16 Summary of Literature Survey

This chapter reviewed gesture recognition and its classification, the main surveys and papers related to the project, and the technologies used in GestureSpeak: OpenCV for image acquisition, MediaPipe for hand-landmark detection, feature extraction, Random Forest classification, evaluation metrics, sentence mapping, text-to-speech and Tkinter. It concluded with the research gap that motivates the project. The next chapter specifies the requirements and the architecture of the system.

CHAPTER 3

SYSTEM REQUIREMENTS AND ARCHITECTURE

3.1 Introduction

System requirements describe the hardware, software, and operational conditions needed to develop and execute an application. They also identify the functions the application must provide and the quality characteristics expected from it.

GestureSpeak requires a webcam, a Python environment, and the libraries used for image processing, hand-landmark extraction, machine-learning classification, and speech output. It also requires the trained classifier and its label encoder, along with the sentence-mapping CSV file.

The system is divided into functional modules rather than implementing every operation in one large program. This design makes the project easier to understand, test, and maintain. Each module performs a specific task and communicates its result to the next stage.

The principal stages are camera capture, landmark extraction, gesture classification, sentence mapping, and speech generation. Training and data collection support the main application but are not necessarily performed every time the application starts.

This chapter describes the requirements, software dependencies, major components, expected behaviour, and limitations that influence the design of GestureSpeak. Requirements are grouped into hardware, software, functional, non-functional, performance and dataset requirements, followed by the constraints of the system and an overview of its architecture.

3.2 Hardware Requirements

The main hardware requirement is a computer capable of running Python and processing webcam frames. The system does not require a specialised glove or dedicated hand-motion sensor because it uses visual information from a camera.

Minimum practical hardware considerations:

- A computer with a supported operating system.
- A working webcam, either built-in or externally connected.
- Sufficient memory to run Python, OpenCV, MediaPipe, and the classifier.
- Adequate storage for source code, model files, training data, and dependencies.
- A microphone is not required for the gesture input because the application uses the webcam.
- Speakers or headphones are recommended for listening to generated speech.

The processor affects how quickly frames can be processed. Available memory influences whether the application and its dependencies can run comfortably alongside other programs.

Camera resolution and frame quality affect the visual information available to the landmark detector.

A high-end GPU is not necessarily required for the basic application because it uses a conventional classifier on extracted landmark features. The actual requirements should be confirmed by running the project on the intended computer. Table 3.1 lists the hardware configuration recommended for development and demonstration.

Component	Requirement
Processor	Intel Core i3 (or equivalent) or higher
Memory (RAM)	4 GB minimum; 8 GB recommended
Storage	Sufficient free space for the source code, Python environment, dataset and model files
Camera	Built-in or external USB webcam
Audio output	Speakers or headphones
Display	Standard monitor capable of showing the application window
GPU	Not required for the basic application

Table 3.1: Recommended Hardware Configuration

3.3 Software Requirements

Python is the primary programming language used in GestureSpeak. It supports rapid development and provides libraries for computer vision, machine learning, graphical interfaces, and speech generation [19].

The project's main software components include the following. Their purposes are listed in Table 3.2.

Software or library	Purpose
Python 3.10 or compatible version	Main programming language
OpenCV	Webcam capture and image processing
MediaPipe	Hand-landmark detection
scikit-learn	Random Forest classification and model utilities, where used by the code
Tkinter	Graphical user interface
pandas or CSV utilities	Reading and managing structured data, depending on implementation
gTTS	Online text-to-speech option
pyttsx3	Local text-to-speech option
pickle	Loading and saving serialised model objects, where used

Table 3.2: Software and Libraries Used

3.3.1 Python

Python is an interpreted, general-purpose language with a large collection of libraries [19]. It is widely used for machine learning and image processing because programs are short and readable. In this project it is used to write every module: webcam capture, landmark extraction, classification, mapping, speech generation and the graphical interface.

3.3.2 OpenCV and MediaPipe

OpenCV provides the camera interface and the image-processing functions [1], [16], while MediaPipe provides the hand-tracking model that returns the landmark coordinates [2], [17]. Both libraries are installed with the Python package manager and run on common desktop operating systems.

3.3.3 scikit-learn and pickle

scikit-learn provides the Random Forest classifier and utilities such as label encoding and data splitting [7], [18]. The trained classifier and the label encoder are saved to files and loaded again by the application. The pickle module of Python is a common way of saving and loading such objects. A pickle file should only be loaded from a trusted source, because loading it can execute code.

3.3.4 Tkinter, gTTS and pyttsx3

Tkinter provides the windows and controls of the application [20]. gTTS and pyttsx3 provide the two speech-output options [21], [22]. Since gTTS needs an internet connection and pyttsx3 needs locally installed voices, both options are kept available.

The exact versions installed should be recorded in the final report or a requirements file. Library compatibility is important because changes between versions may affect available functions and runtime behaviour.

The project should be tested in a clean Python environment. This helps identify missing dependencies and prevents unrelated packages installed on the computer from hiding setup problems.

3.4 Functional Requirements

Functional requirements define what the system must do. They describe the operations visible to the user and the processing tasks performed internally. The main functional requirements of GestureSpeak are given in Table 3.3.

ID	Requirement	Description
FR-01	Application launch	The application should open its main graphical interface when started correctly.
FR-02	Webcam access	The application should attempt to connect to a webcam and handle camera-unavailable conditions.
FR-03	Hand detection	The application should identify a visible hand and obtain its landmark representation.

ID	Requirement	Description
FR-04	Gesture classification	The extracted features should be passed to the trained model to predict a supported gesture label.
FR-05	Sentence mapping	The predicted label should be matched with the appropriate sentence in the mapping file.
FR-06	Language selection	The application should provide the configured language options, including English, Hindi, and Kannada.
FR-07	Speech output	The selected sentence should be passed to the configured TTS engine when speech output is requested or triggered by the application's logic.
FR-08	Error handling	The system should respond appropriately to missing models, invalid mappings, unavailable cameras, or speech-generation failures.

Table 3.3: Functional Requirements of GestureSpeak

3.5 Non-Functional Requirements

Non-functional requirements describe the quality and operational characteristics of the system.

Usability: The graphical interface should make the main operations understandable. Users should be able to start the application, position their hand, view the recognised output, and understand the language setting without needing to inspect the source code.

Performance: The system should process webcam frames at a rate that makes the interaction practical. Performance depends on the computer, camera settings, landmark-processing time, and speech-generation behaviour.

Reliability: The system should handle temporary detection failures and invalid inputs without crashing. A missing hand should not automatically be interpreted as a valid gesture.

Maintainability: Separate modules should make it possible to modify classification, mapping, or speech generation independently.

Portability: The application should run on operating systems that support the required Python libraries and audio components, subject to compatibility testing.

Extensibility: New gesture classes should be addable by collecting examples, preparing the training data, retraining the classifier, and updating the sentence mapping.

Security and privacy: The application processes live video of a person. Frames should be used only for recognition and should not be stored or transmitted unless the user chooses to do so. When gTTS is used, the sentence text, but not the video, is sent to an online service.

3.6 Performance Requirements

Performance requirements describe how efficiently the application should respond to user input. For a real-time gesture application, an important consideration is the delay between presenting a gesture and receiving the corresponding sentence or spoken output.

The complete response time includes several stages: webcam frame capture, hand-landmark estimation, feature preparation, classifier prediction, sentence lookup, and speech generation. The first stages are repeated while the camera is active, whereas speech generation may occur only when the application decides that a gesture is stable enough to announce.

The project README describes a debounce mechanism in which a gesture must remain stable for approximately one second before it is accepted. This can reduce repeated announcements caused by small variations between consecutive frames. It also introduces a deliberate delay before confirming the gesture.

The README also describes a confidence threshold above 80%. This setting should be documented according to the actual implementation, and it should not be treated as proof that every prediction with that value is correct. A confidence-like score is useful only to the extent that the model's output and calibration support its interpretation.

Actual frame rate, response time, and recognition performance should be measured during testing rather than assumed.

3.7 Dataset and Model Requirements

A supervised gesture classifier requires labelled examples. Each example contains a feature representation of a hand gesture and a label indicating the intended gesture class. In GestureSpeak, the feature representation is based on hand landmarks.

The dataset should contain examples of every supported gesture. It should also include natural variation in hand position, orientation, and users wherever possible. If the training data contains only very similar examples, the classifier may perform well on those examples but fail under different conditions.

The project includes a training-data file and a training script. The script can be used to prepare and train the classifier, including the synthetic-data option documented in the README. Synthetic examples can help with development, but they do not automatically represent real-world hand variation. Performance claims should therefore be based on appropriate real-data testing.

The model and label encoder must be saved and loaded consistently. The label encoder helps maintain the relationship between internal numeric class representations and readable gesture labels. A mismatch between the model, encoder, and mapping file can produce incorrect output even if each file loads successfully.

3.7.1 Gesture Vocabulary

The project README describes 15 predefined gestures. Each gesture has a label used by the classifier and a sentence in each supported language. Table 3.4 is the record of the vocabulary and should be completed from the file `gesture_sentences.csv`.

S.No.	Gesture label	English sentence	Hindi sentence	Kannada sentence
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				

Table 3.4: Gesture Vocabulary and Associated Sentences

3.8 Constraints and Limitations

GestureSpeak operates within the constraints of visual hand recognition. The system requires the hand to be visible enough for the landmark detector to estimate its structure. Severe occlusion, motion blur, poor illumination, or an unusual camera angle may reduce landmark quality.

The system is limited to its predefined gesture vocabulary. A gesture that is not represented in training data may be incorrectly classified as one of the known classes unless an effective rejection mechanism is implemented. The system should not assume that every hand configuration belongs to a valid supported gesture.

Language support is another constraint. A sentence can be stored in the mapping file, but successful speech generation depends on the capabilities of the selected speech engine. Online speech generation may require internet access, whereas local speech generation depends on the installed voices and audio configuration.

The system also depends on the consistency of its software environment. Incompatible library versions, missing model files, or incorrect file paths can prevent the application from running.

These constraints define the conditions under which the prototype should be evaluated and help identify the most valuable future improvements.

3.9 System Architecture Overview

The system architecture describes how information moves through the application. The main processing sequence is:

1. The webcam captures a video frame.
2. OpenCV reads and prepares the frame.
3. MediaPipe detects the hand and extracts landmarks.
4. The application converts landmarks into the feature format expected by the classifier.
5. The Random Forest model predicts a gesture label.
6. The application applies its stability or debounce logic.
7. The sentence mapper retrieves the corresponding sentence.
8. The selected language determines which text is displayed and sent to the speech component.
9. The TTS engine generates audio using the configured speech method.

The training workflow is separate from the live workflow. Training data is prepared and supplied to the training script, which creates the model and label encoder. The live application loads these saved files to make predictions.

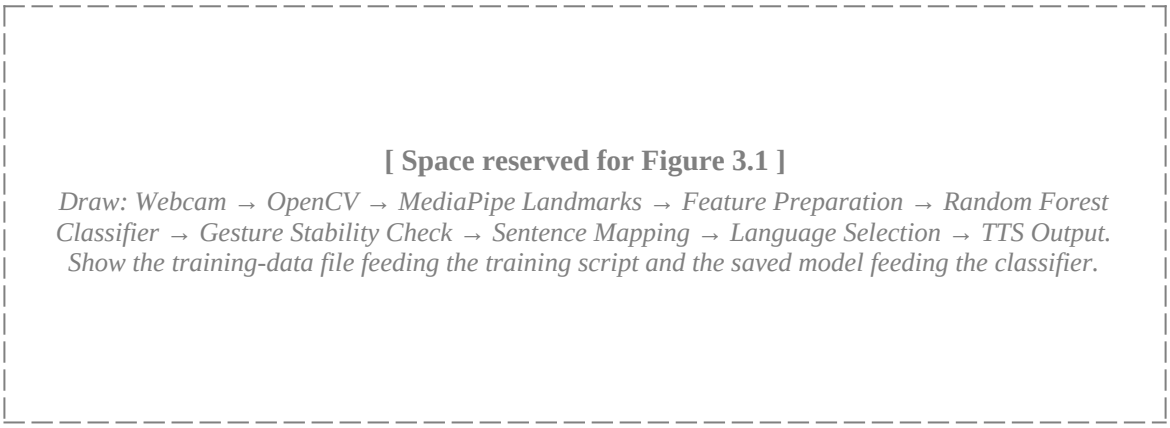


Figure 3.1: Block diagram of the GestureSpeak processing pipeline

3.10 Summary of System Requirements

This chapter specified the hardware and software required by GestureSpeak, listed its functional and non-functional requirements, described the dataset and model requirements, and identified the constraints under which the prototype operates. It ended with an overview of the processing pipeline. The next chapter explains how these requirements are realised in the design and implementation.

CHAPTER 4

SYSTEM DESIGN AND IMPLEMENTATION

4.1 Introduction

System design converts the requirements into a structure that can be implemented as software modules. It defines the responsibility of each component, the order in which operations occur, and the way information is transferred between modules.

GestureSpeak uses a modular design. The camera-processing component is responsible for obtaining visual input, the landmark component represents the hand, and the classifier predicts the gesture category. The sentence mapper converts the category into a message, and the speech component generates audio.

The graphical interface coordinates these operations and presents the results. The model-training and data-collection components support the development lifecycle by allowing training examples to be created and the classifier to be updated.

A modular design has several advantages. Errors can be investigated within individual components, alternative implementations can be introduced more easily, and the system can be extended without rewriting the complete application. For example, the speech engine can be changed without changing how MediaPipe extracts landmarks, provided the interface between the modules remains consistent.

4.2 Architectural Design

The architecture contains two related workflows: the training workflow and the runtime workflow.

During training, labelled gesture features are prepared and passed to the training script. The classifier learns the relationship between the feature vectors and gesture labels. The trained model and the label encoder are then saved for later use.

During runtime, the application loads the saved model and encoder. It obtains frames from the webcam, extracts landmarks, creates the required feature vector, and requests a prediction. The resulting label is passed to the sentence mapper. Once a valid and sufficiently stable gesture has been identified, the corresponding sentence is displayed and sent to the speech component.

The runtime workflow should not retrain the classifier for every webcam frame. Model training is a separate operation because it involves fitting the model to labelled examples. Live inference uses the saved model to predict labels for new input.

[Space reserved for Figure 4.1]

Insert a diagram showing the training path on one side and the runtime path on the other. Connect the saved model and label encoder from the training path to the runtime classifier.

Figure 4.1: Architectural design of GestureSpeak

The architecture can be understood as four layers. The **input layer** consists of the webcam and the OpenCV capture code. The **perception layer** consists of MediaPipe and the feature-preparation step. The **decision layer** consists of the classifier, the confidence threshold and the stability check. The **output layer** consists of the sentence mapper, the language selection, the interface display and the speech engine. Each layer depends only on the output of the layer below it, which keeps the interfaces between the modules small.

4.3 Module Description

The major modules of GestureSpeak are listed in Table 4.1.

Module / file	Main responsibility
main_app.py	Entry point and graphical application
capture.py	Webcam capture and frame preparation
landmark_extraction.py	Extraction of hand landmarks
gesture_classifier.py	Gesture prediction using the trained classifier
sentence_mapper.py	Retrieval of the sentence associated with a gesture
tts_engine.py	Speech generation using configured TTS options
data_collection.py	Collection of gesture examples
train_model.py	Preparation/training workflow for the classifier
gesture_sentences.csv	Gesture-to-sentence mapping across languages
gesture_model.pkl	Saved classifier, subject to the project implementation
label_encoder.pkl	Saved class-label encoder, subject to the project implementation

Table 4.1: Modules and Files of GestureSpeak

Each module should have a clearly defined responsibility. A module should accept input in a known format, perform its assigned operation, and return a result that the next module can use.

This structure reduces the risk of duplicated logic. It also supports unit testing because individual components can be tested separately before the complete application is tested.

4.4 Data Flow Diagrams

A data flow diagram (DFD) shows how data moves between external entities, processes and data stores. The level-0 diagram (context diagram) treats the whole system as one process and shows only its inputs and outputs. In GestureSpeak, the user supplies hand gestures and a language choice, and the system returns the recognised sentence and speech output.

[Space reserved for Figure 4.2]

Draw one central process 'GestureSpeak System'. Inputs: hand gesture (via webcam) and language selection from the User. Outputs: recognised sentence (text) and speech output to the User.

Figure 4.2: Level-0 data flow diagram (context diagram)

The level-1 diagram divides the system into its main processes: capture and landmark extraction, gesture classification, stability check, sentence mapping and speech generation. It also shows the data stores used by the system: the trained model and encoder, and the gesture-sentence CSV file.

[Space reserved for Figure 4.3]

Draw the processes P1 Capture and Extract Landmarks, P2 Classify Gesture, P3 Check Stability, P4 Map Sentence, P5 Generate Speech; data stores D1 Model and Encoder, D2 Sentence Mapping File; and the User as the external entity.

Figure 4.3: Level-1 data flow diagram

4.5 Use-Case Diagram

The use-case diagram represents the interaction between the actors and the system. The **User** is the primary actor, who starts the application, selects the language, shows gestures to the camera, reads the displayed sentence and listens to the speech output. The **Developer** is a second actor who collects gesture samples, retrains the model and edits the sentence mapping file.

[Space reserved for Figure 4.4]

Draw the actors User and Developer and the use cases: Start Application, Select Language, Show Gesture, View Recognised Sentence, Listen to Speech, Collect Gesture Data, Train Model, Edit Sentence Mapping.

Figure 4.4: Use-case diagram of GestureSpeak

4.6 Sequence of Operations

The sequence diagram in Figure 4.5 shows the order of the messages exchanged by the components during one recognition cycle. The interface requests a frame from the capture module, the frame is passed to the landmark extractor, the landmark features are passed to the classifier, and the predicted label is passed to the stability check. When the gesture is accepted, the sentence mapper returns the sentence for the selected language, which is displayed and passed to the speech engine.

[Space reserved for Figure 4.5]

Draw lifelines: User, Interface, Capture, Landmark Extractor, Classifier, Stability Check, Sentence Mapper, TTS Engine, and the messages in the order described above.

Figure 4.5: Sequence diagram for one recognition cycle

4.7 Webcam Capture and Frame Processing

The webcam capture component obtains images from the camera. In OpenCV, a video-capture object is commonly used to connect to the camera and retrieve frames. The program repeatedly reads a frame while the application is running.

The capture module must check whether the camera was opened successfully. It should also verify that each frame was read correctly. If a frame is unavailable, the program should avoid passing invalid data to the landmark detector.

Frame preparation may include resizing, colour-space conversion, or other transformations required by the selected computer-vision component. These operations must be consistent with the expected input format. For example, some components use RGB images while OpenCV commonly reads frames in BGR format.

The camera loop should be designed so that the user can exit the application cleanly. Camera resources should be released when processing ends. Failure to release resources may prevent another application from using the webcam until the process closes.

[Space reserved for Figure 4.6]

Insert a screenshot showing the webcam frame or the application window displaying the live camera feed.

Figure 4.6: Live webcam frame in the application window

4.8 Hand Landmark Extraction

The landmark-extraction module uses MediaPipe to estimate important points on the hand. These points represent the geometric structure of the hand and provide input features for the classifier.

When a hand is detected, the system obtains the landmark information and prepares it in the expected order. The landmark list must be consistent because the classifier assigns meaning to each feature position based on the training process.

If the system fails to detect a hand, it should handle the condition explicitly. A missing detection should not be converted into an arbitrary gesture label. Instead, the application can wait for the hand to become visible or display a message asking the user to position the hand clearly.

The quality of landmark extraction affects every later stage. If the detector estimates a fingertip or joint incorrectly, the feature vector may differ from the actual hand configuration. This can cause the classifier to predict the wrong class even when the model itself is loaded correctly.

[Space reserved for Figure 4.7]

Insert an image of a detected hand with the landmarks and connections visible. Use a screenshot from the running project if possible.

Figure 4.7: Detected hand with landmarks and connections

4.9 Feature Preparation

After landmarks are extracted, they must be converted into the format expected by the classifier. Machine-learning models typically require a numerical feature vector rather than an arbitrary list of visual objects.

The feature-preparation step must preserve the order and meaning of the values used during training. If the model was trained with a fixed sequence of landmark coordinates, live predictions must use the same sequence. Changing the order can make a valid model appear to perform poorly because each feature position would represent a different measurement.

Each of the 21 landmarks provides x, y and z values, so a vector built from all coordinates contains one entry per coordinate in a fixed order (Section 2.7.2). The features may be the coordinates of the hand landmarks themselves or derived geometric measurements, such as the normalised coordinates of Equation 2.1. The method adopted must be identical in the data-collection module, the training script and the live application.

The prepared vector is then passed to the classifier. Before prediction, the program should ensure that the number of values and their general format are compatible with the trained model.

Consistent feature preparation is one of the most important requirements for reliable inference.

4.10 Gesture Classification

The gesture-classification module loads the trained Random Forest model and uses it to predict the class associated with a new feature vector.

The model was trained using labelled examples. During training, it learned patterns that distinguish the gesture categories. During live execution, it applies the learned relationships to the incoming landmark features and produces a predicted label.

The predicted label is an intermediate result, not yet the final sentence. It must be interpreted through the label encoder and passed to the sentence mapper. If the model output is numeric, the encoder may be required to convert it into a readable class label.

The module should also handle invalid conditions, such as a missing model file, incompatible features, or an unexpected prediction. Such failures should be reported clearly so that they can be corrected during development.

The quality of classification depends on the training data, feature representation, model configuration, and similarity between training examples and real-world input.

4.11 Confidence Threshold and Gesture Stability

GestureSpeak's project description specifies a confidence threshold above 80% and a debounce mechanism that requires a gesture to remain stable for approximately one second. These mechanisms are intended to reduce accidental or repeated output.

A confidence threshold is used to reject predictions that do not meet the selected criterion. Its interpretation depends on how the model's confidence or probability value is obtained (Section 2.10.2). A value above a threshold does not guarantee correctness, so the threshold must be evaluated using test data.

Debouncing addresses another problem: predictions can change slightly between consecutive frames, even when the user is trying to hold one gesture. If every frame triggered speech, the application could repeat a sentence many times or speak incorrect intermediate predictions.

A stability check waits until the same gesture has been recognised consistently for the required interval before accepting it. The implementation must also define when the system is allowed to announce the same gesture again, otherwise the same held gesture may trigger repeated speech.

[Space reserved for Figure 4.8]

Draw: Prediction → Confidence above threshold? (No → continue monitoring) → Same gesture held for about one second? (No → continue monitoring) → Accept gesture → Display and speak sentence.

Figure 4.8: Flowchart of the confidence threshold and stability check

4.12 Sentence Mapping

The sentence-mapping module connects a predicted gesture label with its predefined text. In GestureSpeak, the mapping data is stored in `gesture_sentences.csv`, which includes sentence versions for English, Hindi, and Kannada according to the project description.

Using a separate mapping file keeps the vocabulary independent of the classifier. The classifier identifies a label, while the mapping component determines the message associated with that label. This means a sentence can be edited without retraining the classifier, provided the underlying label remains unchanged.

The mapping file must be structured consistently. Column names should be valid, labels should match the classifier's output, and each supported gesture should have the required

language values. Missing translations should be handled explicitly rather than silently returning incorrect text. Table 4.2 shows the logical structure of the file.

Gesture label	English sentence	Hindi sentence	Kannada sentence
label used by the classifier	sentence in English	sentence in Hindi (UTF-8)	sentence in Kannada (UTF-8)

Table 4.2: Logical Structure of the Sentence Mapping File

The project description mentions 15 predefined gestures, while the CSV may contain multiple language entries for each gesture. The number of CSV rows should not automatically be treated as the number of unique gestures because a file may store one row per gesture or use another representation.

4.13 Multilingual Sentence Selection

Multilingual output allows the same recognised gesture to produce a sentence in different languages. GestureSpeak supports English, Hindi, and Kannada sentence mappings according to its project description.

The language-selection process should take place after the gesture label has been identified. The mapping module retrieves the selected language's sentence for that label. The sentence is then displayed and passed to the TTS component.

The mapping and speech engine must be considered together. A sentence may be available in the CSV but may not be spoken correctly if the selected TTS engine lacks the required voice or language support. This is particularly important when testing Indian-language output across different computers.

A robust implementation should provide clear feedback when a translation is missing or speech generation is unavailable. The program should not replace a missing translation with a sentence from a different language without informing the user.

The separation of gesture recognition and language selection makes it possible to expand the vocabulary without redesigning the complete classification pipeline.

4.14 Text-to-Speech Implementation

The text-to-speech module converts the selected sentence into audio. Its input is the final text from the sentence mapper, and its output is spoken audio produced through the configured speech engine.

GestureSpeak includes gTTS and pyttsx3 options. gTTS commonly requires internet connectivity because it uses an online service. pyttsx3 relies on local speech engines and installed voices. The actual fallback behaviour depends on how the project code invokes these components and handles errors.

The speech module should separate text preparation from audio generation. This makes it possible to check whether the correct sentence was selected before investigating an audio problem. It also prevents the speech component from needing direct access to webcam frames or landmark data.

Audio output may take longer than classification. If speech generation blocks the main camera-processing loop, the visual interface may appear temporarily unresponsive. The implementation should be tested to determine whether this occurs and whether asynchronous execution is necessary (Section 2.14).

[Space reserved for Figure 4.9]

Insert a screenshot of the interface showing a recognised sentence and the selected language.

Figure 4.9: Interface showing a recognised sentence and the selected language

4.15 Graphical User Interface

The graphical user interface provides the main point of interaction between the user and GestureSpeak. The project uses Tkinter according to its architecture description. A GUI can display instructions, language options, recognition results, and application status.

A clear interface should communicate the application's current state. For example, it may indicate whether the webcam is active, whether a hand has been detected, which gesture was predicted, and whether speech output is being generated. These status messages help users understand the system's behaviour without inspecting the console.

The interface should avoid displaying a predicted sentence as confirmed when the gesture has not passed the configured stability checks. It should also communicate camera or speech errors in understandable language.

A simple interface is valuable for academic demonstration because the evaluator can observe the input and output in one place. It also makes testing easier: the tester can provide a known gesture and compare the displayed result with the expected result.

[Space reserved for Figure 4.10]

Insert a screenshot of the actual running GestureSpeak interface (label it clearly if it is not a screenshot of the final implementation).

Figure 4.10: Main window of the GestureSpeak application

4.16 Data Collection and Model Training

Data collection and model training support the addition and improvement of gesture classes. The data-collection component records examples in the format required by the training workflow. Each example must be associated with the correct label so that the classifier learns the intended relationship between features and classes.

The training script prepares the data, fits the classifier, and saves the resulting model and label encoder. The README documents a synthetic-training option using the following command:

```
python train_model.py --synthetic
```

This command should be run only in accordance with the project's actual implementation and environment.

Synthetic training data can help demonstrate that the training pipeline operates, but its performance may not reflect real-world use. A meaningful evaluation should use representative examples captured under conditions similar to the intended application. Training and test examples should be separated to reduce the risk of reporting performance on examples the model has already seen.

After retraining, the saved model, encoder, and sentence mappings must remain compatible. The application should be retested after any change to the gesture vocabulary or feature representation. The general training procedure is summarised in Algorithm 4.1.

```
START
   Load the file of labelled landmark feature vectors
   Separate the feature vectors X and the gesture labels y
   Encode the labels y into numeric classes using the label encoder
   Split the data into a training set and a test set
   Create a Random Forest classifier
   Fit the classifier on the training set
   Evaluate the classifier on the test set
   Save the trained model and the label encoder to files
END
```

Algorithm 4.1: General procedure for training the gesture classifier

4.17 Runtime Algorithm and Implementation Summary

The overall runtime algorithm can be described as shown in Algorithm 4.2.

```
START
   Load the trained gesture model and label encoder
   Load the gesture-to-sentence mapping
   Initialise the graphical interface and webcam
   WHILE the application is running:
      Capture a webcam frame
      IF frame capture fails:
         Display or handle the error
         Continue or exit safely
      Detect the hand using MediaPipe
      IF no hand is detected:
         Continue monitoring
      Extract the landmarks
      Prepare features in the required format
      Predict the gesture label
      Apply the configured threshold and stability logic
      IF a gesture is accepted:
         Retrieve the sentence in the selected language
         Display the sentence
         Generate speech using the configured TTS engine
   Release the webcam and close resources
END
```

Algorithm 4.2: Runtime algorithm of GestureSpeak

The application is started with the documented entry point `main_app.py` (Section 6.2). The model and mapping are loaded once at start-up and not for every frame, because loading files is slow compared with prediction. Within the loop, the stages are executed in a fixed order, and any stage that cannot produce a valid result stops the processing of the current frame and returns control to the capture step.

4.18 Error Handling Strategy

Because the system depends on hardware (camera and audio), external files (model, encoder, mapping) and, for gTTS, an online service, it must handle several error conditions. Table 4.3 lists the main conditions and the expected behaviour of the application.

Condition	Where detected	Expected handling
Webcam cannot be opened	Capture module	Show a clear message; do not start recognition
Frame not received	Capture module	Skip the frame; exit safely if the failure persists
No hand visible	Landmark extractor	Continue monitoring; do not output a gesture

Condition	Where detected	Expected handling
Model or encoder file missing	Classifier module	Report the problem before attempting prediction
Label without mapping entry	Sentence mapper	Report the missing entry; do not send empty text to the speech engine
Missing translation	Sentence mapper	Inform the user instead of silently using another language
Speech generation fails	TTS module	Show the sentence on screen and report the speech error

Table 4.3: Error Conditions and Expected Handling

4.19 Summary of System Design and Implementation

This chapter described the modular design of GestureSpeak: its architecture, module structure, data flow, use cases and sequence of operations. It then explained the working of each stage: webcam capture, landmark extraction, feature preparation, classification, confidence threshold and stability check, sentence mapping, multilingual selection, speech generation, the graphical interface, and data collection and training. The runtime and training algorithms and the error-handling strategy complete the design. The next chapter describes how the system is tested.

CHAPTER 5

SOFTWARE TESTING

5.1 Introduction

Software testing is the process of checking whether an application behaves as expected under specified conditions. It helps identify defects, incorrect outputs, integration problems, and situations that the program does not handle properly.

GestureSpeak includes several connected components, so testing must cover both individual modules and the complete workflow. A successful webcam connection does not prove that gesture classification is correct. Similarly, a correct predicted label does not guarantee that the mapping file contains the intended sentence or that the TTS engine can speak it.

Testing should therefore examine the application from multiple perspectives: whether each function works independently, whether modules communicate correctly, and whether the complete application produces the expected output.

The testing process should include valid input, invalid input, missing components, and difficult operating conditions. The results should be recorded in a table with test identifiers, input conditions, expected outcomes, actual outcomes, and status.

The report should include only tests that were actually executed. Any test not yet performed should be marked as pending rather than described as passed.

5.2 Testing Objectives

The primary objective of testing is to verify that GestureSpeak performs its intended operations correctly and handles common errors safely.

The specific objectives are:

1. To verify that the application launches successfully in the configured environment.
2. To verify that the webcam opens and provides usable frames.
3. To check that hand landmarks are extracted when a hand is visible.
4. To verify that landmark features are passed to the classifier in the expected format.
5. To confirm that the classifier produces a supported gesture label for known test inputs.
6. To verify that gesture labels map to the correct sentences.
7. To test language selection for English, Hindi, and Kannada.
8. To verify that the configured TTS engine produces audio where supported.
9. To check the behaviour of the threshold and stability mechanisms.
10. To identify failures caused by missing files, invalid mappings, unavailable hardware, or network problems.

These objectives cover the major stages of the application and provide a structured way to evaluate its correctness.

5.3 Levels and Types of Testing

Testing is commonly described using two related terms. **Verification** checks whether the software has been built correctly, that is, whether it conforms to its design and specification. **Validation** checks whether the right software has been built, that is, whether it satisfies the needs of the user. Both are needed: a program can follow its design exactly and still be unsuitable for the intended use.

Two broad testing techniques are used. In **black-box testing**, the tester supplies inputs and examines the outputs without looking at the internal code. Functional testing of GestureSpeak, in which a gesture is shown and the sentence is observed, is of this type. In **white-box testing**, the tester uses knowledge of the internal structure to design the tests, for example to make sure that every branch of the error-handling code is executed. Unit tests of the individual modules are usually of this type.

Testing is also organised in levels. **Unit testing** examines individual functions or modules. **Integration testing** examines the interaction between modules. **System testing** examines the complete application against its requirements. **Acceptance testing** checks whether the user finds the system suitable. Performance and usability tests examine non-functional qualities. The following sections describe how these levels apply to GestureSpeak.

[Space reserved for Figure 5.1]

Insert a diagram of testing levels as a pyramid or V-model: Unit Testing → Integration Testing → System / Functional Testing → Usability and Acceptance Testing.

Figure 5.1: Levels of testing applied to GestureSpeak

5.4 Unit Testing

Unit testing checks individual functions or modules independently. It helps locate defects before the modules are combined into the complete application.

For GestureSpeak, the sentence mapper can be tested with known gesture labels and expected sentence values. The classifier-loading function can be checked to determine whether the model file is found and loaded. The feature-preparation function can be tested using a known landmark representation to confirm that it returns the expected structure.

The TTS module can be tested using a simple sentence. The objective is to determine whether the configured engine accepts the text and generates audio in the current

environment. A failure may indicate an engine configuration problem rather than a gesture-recognition problem.

Unit tests should be repeatable and should not depend unnecessarily on a live webcam. Where possible, known test inputs should be used so that the same operation can be repeated under controlled conditions.

Unit testing does not prove that the entire application works, but it reduces the number of possible failure points when integration testing begins.

5.5 Integration Testing

Integration testing verifies whether the individual modules work correctly when connected. GestureSpeak depends on the correct transfer of data from one stage to another.

The first integration check is the connection between webcam capture and landmark extraction. A valid frame should be passed in the correct format to the hand-detection component. The next check verifies that the landmark output is converted into features compatible with the trained classifier.

The classifier's output must then be interpreted correctly by the label encoder and passed to the sentence mapper. The mapper should return the sentence associated with the predicted gesture and selected language. Finally, the sentence should be accepted by the TTS component.

Integration testing should also examine error propagation. For example, if a mapping is missing, the application should not silently send an empty or unrelated sentence to the speech engine. If the model cannot be loaded, the program should report the problem before attempting prediction.

The purpose is to confirm that the complete data flow is correct, not merely that each module can run independently.

5.6 Functional and Usability Testing

Functional testing verifies that the system's features match the requirements. In GestureSpeak, this includes launching the application, starting webcam processing, recognising supported gestures, selecting a language, displaying a sentence, and generating speech.

Each feature should be tested with an expected result. For example, when a valid gesture is accepted, the displayed sentence should match the mapping file for that label and language. If the gesture is not detected, the application should not generate an arbitrary sentence.

Usability testing examines whether a person can understand and operate the application. The interface should provide clear instructions and feedback. A user should be able to

determine whether the camera is active, whether a hand is detected, and which sentence has been recognised.

The application should also provide a clear way to stop the webcam and close the interface. If an error occurs, the message should help the user identify the problem instead of displaying only a technical traceback.

Functional testing focuses on correctness, whereas usability testing focuses on how effectively a user can interact with the application.

5.7 Performance Testing

Performance testing is performed to evaluate the response and processing capability of the system. The time required for stages such as frame capture, landmark extraction, classification and result display is considered. The quantities to be measured are described below, and the measured values should be recorded after execution on the target computer.

Three quantities are of main interest. The **frame rate** is the number of frames processed per second while the camera is active. The **processing latency** is the time between capturing a frame and obtaining its predicted label. The **confirmation delay** is the time between the start of a held gesture and the announcement of its sentence, which includes the deliberate stability interval of about one second. Table 5.1 is used to record these measurements.

Quantity	Method of measurement	Measured value
Frame rate (FPS)	Count processed frames over a fixed time interval	
Processing latency per frame	Time from frame capture to predicted label	
Confirmation delay	Time from showing a gesture to display of the sentence	
Speech start delay	Time from displayed sentence to start of audio	

Table 5.1: Performance Measurements

5.8 Test Environment

The test environment should be recorded, because the same application may behave differently on another computer. Table 5.2 records the details to be noted before the tests are run.

Item	Details
Operating system and version	
Python version	
OpenCV, MediaPipe and scikit-learn versions	
Webcam model and resolution	
Processor and memory	

Item	Details
Lighting conditions	
TTS engine used / internet availability	

Table 5.2: Test Environment Details

5.9 Test Cases

Table 5.3 provides a test plan. The actual output and result columns are to be filled after executing each case, using Pass, Fail, or Not Executed.

Test ID	Test condition	Expected result	Actual output	Result
TC-01	Launch the application with dependencies installed	Main interface opens		
TC-02	Start the application with a working webcam	Camera frames are received		
TC-03	Start with webcam unavailable	Clear error or safe handling		
TC-04	Present a visible supported gesture	Hand landmarks are detected		
TC-05	Present a gesture that remains stable	Classifier output is evaluated by stability logic		
TC-06	Select English output	English sentence is retrieved		
TC-07	Select Hindi output	Hindi sentence is retrieved		
TC-08	Select Kannada output	Kannada sentence is retrieved		
TC-09	Generate speech for a mapped sentence	Configured TTS engine handles the sentence		
TC-10	Use an invalid or missing mapping	Error is handled without unrelated output		
TC-11	Remove or rename the model file for a controlled test	Model-loading error is reported		
TC-12	Close the application	Camera resources are released		
TC-13	Show a gesture for less than the stability interval, then change it	No sentence is announced for the short gesture		
TC-14	Present an ambiguous or unsupported hand pose	Prediction below the threshold is not accepted		

Test ID	Test condition	Expected result	Actual output	Result
TC-15	Remove the hand from the camera view	No gesture is output; system keeps monitoring		
TC-16	Test under low or uneven lighting	Behaviour is observed and recorded		

Table 5.3: Test Cases and Test Results

The table is a test plan, not evidence that all tests have passed. Test cases TC-13 to TC-16 examine the behaviour of the threshold, the stability check and the operating conditions that the project description identifies as sources of error.

5.10 Testing Strategy and Summary

A systematic testing strategy begins with individual modules and progresses toward the complete application. Unit tests check the isolated functions, integration tests verify the flow of data between modules, and functional tests verify the user-visible requirements.

Testing should include both normal and abnormal conditions. Normal cases include a well-lit hand, a supported gesture, a valid mapping, and an available speech engine. Abnormal cases include no visible hand, an unavailable camera, a missing model file, a label that has no mapping, and an unavailable online TTS service.

The test environment should be recorded, including the Python version, operating system, library versions, camera used, and whether the speech engine required internet access. These details help explain why the same application may behave differently on another computer.

Chapter summary: This chapter described the testing objectives, the levels and types of testing, unit testing, integration testing, functional and usability testing, performance testing, the test environment and a practical test-case table. Final conclusions about reliability should be based on the recorded outcomes rather than assumed results.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Introduction

The results and discussion chapter explains what the developed application produces when it is executed. It should connect the project's objectives with observable outputs and the evidence collected during testing.

For GestureSpeak, the primary output is a sentence corresponding to a recognised hand gesture. The system also provides speech output through the configured TTS component. The recognition pipeline includes webcam capture, landmark extraction, classification, sentence mapping, and audio generation.

The results should be demonstrated through screenshots of the actual application. Suitable evidence includes the main interface, the webcam view, the recognised gesture, the displayed sentence, and the selected language. Screenshots should be accompanied by explanations describing the input condition and the output observed.

The results chapter must distinguish between intended behaviour and measured performance. A screenshot can demonstrate that a feature is available, but it does not establish overall classification accuracy. Recognition accuracy requires a labelled test dataset and a documented evaluation procedure.

6.2 Application Execution

The application is started using the project environment and the documented entry point:

```
python main_app.py
```

Before execution, the required libraries and project files must be available. The trained model and label encoder must be present at the paths expected by the program. The webcam must also be accessible to the Python process.

When the application starts correctly, the user should be able to interact with the graphical interface and initiate the configured webcam-based workflow. If the application fails to start, the error should be investigated before recognition results are evaluated.

A successful launch is the first stage of functional verification. The next stages are confirming camera capture, hand detection, classification, sentence mapping, and speech output.

[Space reserved for Figure 6.1]

Insert a screenshot of the GestureSpeak application after successful launch. Identify the visible interface components in the caption or accompanying text.

Figure 6.1: GestureSpeak application after successful launch

6.3 Webcam and Landmark Detection Results

The webcam stage supplies the live visual input required by the system. The user places a hand within the camera's field of view, and the application processes the resulting frames.

Landmark detection is an important intermediate result because the classifier depends on the representation produced by this stage. If the hand is not detected, the system cannot create the expected feature vector. The application should therefore distinguish between a valid detected hand and a frame in which no usable landmarks are available.

The detection stage should be tested under different conditions, including changes in hand position and lighting. The purpose is to identify conditions in which landmark quality decreases. The observed results should be documented with screenshots or notes from actual execution.

[Space reserved for Figure 6.2]

Insert a screenshot showing a hand with the detected landmark points. If landmarks are not shown in the final interface, capture them from a debugging or demonstration view and label the figure accurately.

Figure 6.2: Hand with detected landmark points

6.4 Gesture Classification Results

After landmark extraction and feature preparation, the classifier predicts a gesture label. The application then uses that label to retrieve the associated sentence.

A correct prediction depends on the relationship between the live feature vector and the examples used to train the model. If the gesture is represented well in the training data, the classifier may distinguish it from other supported classes. If two gestures have similar configurations or the live hand differs from the training examples, the classifier may produce an incorrect label.

A useful result demonstration should show the input gesture, the predicted label, and the corresponding sentence. It should also identify the test condition, such as the lighting environment and whether the gesture was held steadily.

[Space reserved for Figure 6.3]

Insert a screenshot of an actual gesture prediction showing the gesture, the predicted label and the sentence.

Figure 6.3: Gesture prediction and recognised label

One or two successful demonstrations are evidence of functional operation, not a complete evaluation of accuracy across all gesture classes.

6.5 Sentence Mapping and Language Results

The sentence-mapping stage converts a predicted gesture label into a message that the user can understand. The mapping file separates sentence content from the classification model and stores the configured language alternatives.

Testing should confirm that a known gesture label retrieves the correct sentence. The test should be repeated for each supported language. If the mapping file contains an empty translation or a label that does not match the classifier output, the application may fail to display the intended message.

[Space reserved for Figure 6.4]

Insert a screenshot of a recognised gesture with the language set to English.

Figure 6.4: Sentence displayed with English output selected

[Space reserved for Figure 6.5]

Insert a screenshot of the same gesture with the language set to Hindi (Devanagari script).

Figure 6.5: Sentence displayed with Hindi output selected

[Space reserved for Figure 6.6]

Insert a screenshot of the same gesture with the language set to Kannada.

Figure 6.6: Sentence displayed with Kannada output selected

Table 6.1 can be used to record the actual observations. The gesture labels and sentences must be taken from `gesture_sentences.csv`.

Gesture label	Selected language	Expected sentence	Actual sentence
	English		
	Hindi		
	Kannada		
	English		
	Hindi		
	Kannada		

Table 6.1: Sentence Mapping Observations

6.6 Speech Output Results

Speech output is the final communication stage of GestureSpeak. After a sentence is retrieved, the TTS module receives the text and attempts to generate audio.

The speech output should be evaluated separately from gesture classification. If the correct sentence appears on the screen but no sound is produced, the problem may lie in the TTS engine, language voice, network connection, or audio device rather than the classifier.

gTTS may require an internet connection, while pyttsx3 depends on local speech-engine availability. The report should describe which engine was actually used during the test and whether a fallback mechanism operated successfully.

The system should be tested using short sentences in each supported language. The tester should verify whether audio is generated, whether the language is appropriate, and whether the output can be heard through the selected device.

[Space reserved for Figure 6.7]

Insert a screenshot showing the recognised sentence and language selection while speech is generated. If the audio itself cannot be shown, describe the observed speech result in the accompanying text.

Figure 6.7: Recognised sentence during speech output

Language	TTS engine used	Audio generated (Yes/No)	Remarks
English			
Hindi			
Kannada			

Table 6.2: Speech Output Observations

6.7 Recognition Performance Evaluation

Classification performance should be measured using labelled test examples that were not used to train the classifier. Each test example has a known correct gesture label, allowing the predicted label to be compared with the expected one.

Accuracy is one common metric, and is calculated as in Equation 6.1.

(6.1)

For example, if 90 predictions are correct out of 100 independently evaluated examples, the accuracy is 90%. This is an illustration of the calculation, not a reported result for GestureSpeak.

Accuracy alone may not reveal which gestures are frequently confused. A confusion matrix shows how the predicted classes correspond to the true classes. Precision and recall can also be calculated for each class when a more detailed evaluation is required (Section 2.11).

The report should state the size and source of the test set, how it was separated from training data, and which gesture classes were evaluated. If no independent test has been performed, the report should say that measured recognition accuracy is not yet available. Table 6.3 and Figure 6.8 are provided for recording the measured results.

Gesture label	Test samples	Precision	Recall	F1-score

Gesture label	Test samples	Precision	Recall	F1-score
Overall accuracy				

Table 6.3: Per-Class Recognition Performance

[Space reserved for Figure 6.8]

Insert the confusion matrix produced from the test set (rows: true gestures, columns: predicted gestures).

Figure 6.8: Confusion matrix of the trained classifier on the test set

6.8 Discussion

The results of a gesture-recognition prototype must be interpreted with the conditions of the test in mind. Several factors influence the behaviour of GestureSpeak.

- **Landmark quality:** the classifier receives only the landmarks produced by MediaPipe. If the detector places a landmark incorrectly because of occlusion, blur or poor lighting, the classifier receives a feature vector that does not correspond to the real hand.
- **Similar gestures:** two gestures whose landmark patterns are close to each other are more likely to be confused. The confusion matrix identifies such pairs, and the remedy is to collect more varied examples or to change the gestures.
- **Training data:** performance on users and conditions that are not represented in the training data may be lower than on the data used for training. Results obtained with synthetic data demonstrate the training pipeline but do not represent real hand variation.
- **Threshold and stability settings:** a higher confidence threshold and a longer stability interval reduce wrong announcements but make the system slower and may

reject correct gestures; lower values do the opposite. The values should therefore be chosen after observing the test results.

- **Speech engine:** the quality, availability and delay of the audio depend on the selected engine, the installed voices and, for gTTS, the network.

Any measured result, such as a frame rate or an accuracy figure, should be stated together with the conditions under which it was obtained.

6.9 Overall Result Analysis

The overall system should be evaluated as a sequence of connected functions rather than by examining classification alone. A gesture may be classified correctly while its sentence mapping is missing, or the correct sentence may be displayed while speech generation fails.

A complete evaluation therefore considers the following:

- Whether the application launches correctly.
- Whether webcam frames are received.
- Whether the hand is detected.
- Whether a supported gesture is classified correctly.
- Whether the stability mechanism prevents unnecessary repeated output.
- Whether the sentence mapping returns the correct language-specific message.
- Whether the TTS engine produces audio in the tested environment.
- Whether the application handles unavailable hardware or missing resources safely.

The main evidence should include the completed test table, application screenshots, and measured classification metrics if they have been calculated. Any known limitations should be discussed openly.

6.10 Summary of Results

Chapter summary: The chapter explains how to present runtime output, recognition behaviour, multilingual mapping, speech generation, and quantitative evaluation. The reserved figures and the tables should be completed with actual screenshots and test observations.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

GestureSpeak demonstrates the integration of computer vision, landmark-based hand representation, machine-learning classification, multilingual sentence mapping, and text-to-speech generation. The application is designed to recognise predefined hand gestures and convert the predicted gesture labels into sentences in English, Hindi, or Kannada.

OpenCV provides webcam input and image-processing support, while MediaPipe extracts hand landmarks. The Random Forest classifier predicts the gesture category, and the sentence-mapping component retrieves the associated message. The TTS component then attempts to convert the selected text into speech using the configured engine.

The modular structure separates recognition, mapping, and audio generation. This makes the project suitable for academic study because each stage can be examined and improved independently. The project also demonstrates how a trained model can be integrated into an interactive Python application rather than being used only in a standalone experiment.

The prototype's practical effectiveness depends on the quality of its training data, the supported gesture vocabulary, the consistency of landmark features, and the behaviour of the speech engine. Its actual performance should be stated using evidence from testing.

GestureSpeak provides an academic prototype for converting predefined hand gestures into text and spoken sentences. It brings together image acquisition, hand-landmark detection, feature preparation, machine-learning classification, multilingual mapping, and speech generation. The project illustrates how separate technologies can be organised into a single software pipeline, and it demonstrates the importance of consistent data representation, labelled training examples, modular programming, and systematic testing. These principles are applicable to many other computer-vision and machine-learning applications.

7.2 Project Contributions and Limitations

The main contribution of GestureSpeak is the combination of multiple technologies into a single predefined-gesture communication workflow. It provides a practical example of how visual information can be converted into a structured machine-learning input and then transformed into a sentence and spoken output.

The project also demonstrates the value of modular software design. The classifier can be retrained, sentence mappings can be edited, and the speech component can be modified without requiring every module to be rewritten. The data-collection and training components provide a foundation for expanding the gesture vocabulary.

Several limitations remain. The application recognises only gestures represented in its supported classes and training data. Its output may be affected by lighting, occlusion, camera angle, and differences between users. A predicted class may also be wrong, and a confidence threshold does not guarantee correctness.

The application does not provide unrestricted sign-language translation and should not be presented as a complete replacement for established communication support. Its role is to demonstrate a technical approach that can be improved through additional data, testing, and user-centred development. The system has a defined scope and should be evaluated within that scope: it does not recognise every possible hand movement, and its reliability depends on the training data, landmark quality, classifier behaviour, mapping-file consistency, and speech-engine configuration.

7.3 Future Enhancements

GestureSpeak can be extended in several directions.

Expansion of the gesture vocabulary: More gesture classes can be added by collecting representative examples, checking their labels, retraining the model, and updating the sentence mappings.

Improved dataset diversity: Training data from different users, camera angles, lighting conditions, and hand positions can help evaluate whether the classifier generalises beyond the original collection environment.

More robust classification: Alternative classifiers or deep-learning methods can be compared using the same evaluation protocol. The choice should be based on measured performance, computational cost, and ease of deployment.

Continuous gesture recognition: The current predefined-gesture approach can be extended to consider sequences of gestures. This would require temporal modelling, for example with recurrent or attention-based networks, and a way to distinguish a completed message from intermediate movements.

Two-hand and body gestures: MediaPipe can track more than one hand, so gestures that use both hands could be added. Facial expression and body posture could be considered later to provide richer context.

Sentence construction: Instead of mapping one gesture to one fixed sentence, a sequence of recognised gestures could be combined into a longer message, with a simple grammar for each supported language.

Improved language support: Additional languages and more natural sentence mappings can be added, provided suitable text and speech support are available. An offline voice for each supported Indian language would remove the dependence on an internet connection.

Wider deployment: The recognition pipeline could be packaged as a mobile or web application so that it can be used on a smartphone camera, which would improve accessibility.

User-centred evaluation: Testing with intended users can help identify confusing gestures, interface problems, and practical accessibility requirements.

These enhancements should be prioritised according to their feasibility and measured benefit rather than added only for complexity.

7.4 Proposed Development Roadmap

Future development should follow a structured roadmap so that improvements can be tested without disrupting the existing application.

The **first stage** should focus on improving the quality of the current gesture dataset. Every supported gesture should have correctly labelled examples, and the feature format should remain consistent between training and live prediction. The updated model should then be evaluated using examples that were not used during training.

The **second stage** should focus on robustness. The application can be tested under different lighting conditions, camera positions, and hand orientations. These tests can help identify gestures that are difficult to distinguish and conditions that cause landmark detection to fail.

The **third stage** should focus on the user interface and speech output. Language selection, recognition feedback, and error messages should be made clear. The application should also handle cases in which speech generation is unavailable.

The **fourth stage** may investigate sequence-based gesture recognition. This would require a mechanism to interpret a series of gestures and determine when a complete message has been communicated. Such an extension is more complex than recognising individual predefined gestures and should be treated as a separate development objective.

With further dataset development, quantitative evaluation, and usability testing, GestureSpeak can serve as a foundation for a more capable gesture-based communication tool. The project achieves its academic purpose by demonstrating the connection between computer vision and practical human–computer interaction.

REFERENCES

[1] G. Bradski, “The OpenCV Library,” Dr. Dobb’s Journal of Software Tools, 2000.

[2] C. Lugaresi et al., “MediaPipe: A Framework for Building Perception Pipelines,” arXiv preprint arXiv:1906.08172, 2019.

[3] F. Zhang et al., “MediaPipe Hands: On-device Real-time Hand Tracking,” arXiv preprint arXiv:2006.10214, 2020.

[4] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[5] L. Breiman, “Bagging Predictors,” Machine Learning, vol. 24, no. 2, pp. 123–140, 1996.

[6] T. K. Ho, “Random Decision Forests,” in Proc. 3rd Int. Conf. on Document Analysis and Recognition, vol. 1, 1995, pp. 278–282.

[7] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[8] C. Cortes and V. Vapnik, “Support-Vector Networks,” Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.

[9] S. Mitra and T. Acharya, “Gesture Recognition: A Survey,” IEEE Transactions on Systems, Man, and Cybernetics, Part C, vol. 37, no. 3, pp. 311–324, 2007.

[10] S. S. Rautaray and A. Agrawal, “Vision Based Hand Gesture Recognition for Human Computer Interaction: A Survey,” Artificial Intelligence Review, vol. 43, no. 1, pp. 1–54, 2015.

[11] M. J. Cheok, Z. Omar, and M. H. Jaward, “A Review of Hand Gesture and Sign Language Recognition Techniques,” International Journal of Machine Learning and Cybernetics, vol. 10, no. 1, pp. 131–153, 2019.

[12] T. Starner and A. Pentland, “Real-Time American Sign Language Recognition from Video Using Hidden Markov Models,” in Proc. International Symposium on Computer Vision, 1995, pp. 265–270.

[13] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. New York, NY, USA: Pearson, 2018.

[14] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning, 2nd ed. New York, NY, USA: Springer, 2009.

[15] P. Taylor, Text-to-Speech Synthesis. Cambridge, U.K.: Cambridge University Press, 2009.

[16] OpenCV, “OpenCV Documentation.” [Online]. Available: https://docs.opencv.org/

[17] MediaPipe, “Hand Landmarker and Hand Tracking Documentation.” [Online]. Available: https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker

[18] scikit-learn developers, “RandomForestClassifier Documentation.” [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

[19] Python Software Foundation, “Python Documentation.” [Online]. Available: https://docs.python.org/3/

[20] Python Software Foundation, “Tkinter — Python Interface to Tcl/Tk.” [Online]. Available: https://docs.python.org/3/library/tkinter.html

[21] gTTS developers, “gTTS Documentation.” [Online]. Available: https://gtts.readthedocs.io/

[22] pyttsx3 developers, “pyttsx3 Documentation.” [Online]. Available: https://pyttsx3.readthedocs.io/

[23] Project source code and accompanying README for GestureSpeak — Hand Gesture to Sentence & Voice Conversion.